QRM Project 1

Léon Le Bohellec | 342583,
Arthur Stämpfli | 396577,
Kilian Wan | 340838

March 8, 2026

1. Empirical stylized facts.

We have a dataframe with dates as index, going from January 1,2023 to June 30, 2025. This dataframe has three columns: **TICKERS** = {**AAPL**, **META**, **JPM**}, initially with their prices. We thus have P_t^i, with $i \in$ **TICKERS**, the price of asset i at time t.

a) We start by building a log-returns dataframe, where $R_t^i = \log\left(P_t^i / P_{t-1}^i\right)$. We drop the first observation due to the shift. The time series of those log-returns are displayed on Fig. 1.

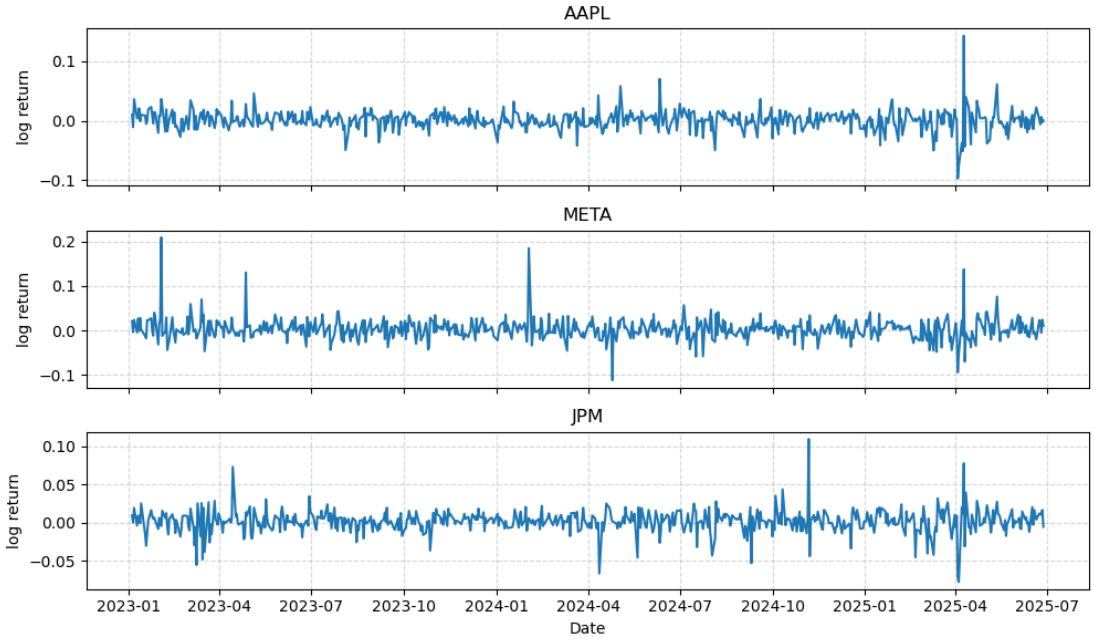

Fig. 1: Return series of the three stocks

Comments. When looking at the return series of AAPL, META, and JPM (Fig. 1), we notice several common features. All three series fluctuate around zero, which is expected for daily log-returns, but they do not behave like white noise.

- AAPL shows long periods where the returns stay small and stable, but these calm phases are interrupted by sudden spikes. These jumps break the apparent stationarity and might suggest that the volatility changes over time rather than staying constant.
- META has even larger movements, with several positive and negative jumps, compared to the other two stocks, which might imply that META has more volatility overall.
- JPM shows less volatility than the two first stocks since its returns fluctuate closer to zero, and the spikes are generally smaller (the y-scale is not the same for the three stocks, we decided to do so in order to see the different volatility clusters).

A common pattern across all assets is that periods of high volatility tend to follow each other, and the same happens for calmer periods. We also notice a volatility cluster around April 2025 common to the three stocks. This spike of volatility in April 2025 coincides with Donald Trump's announcement of new tariffs. This event caused a lot of agitation in the financial markets and thus contributed to the increase in volatility. However, while this is a possible explanation, we can not attribute this spike to this single event based solely on this data

Let's now plot the volatility trends (using a rolling window of 21, which simulates a trading month) to see whether the pattern observed before is really seen or not.

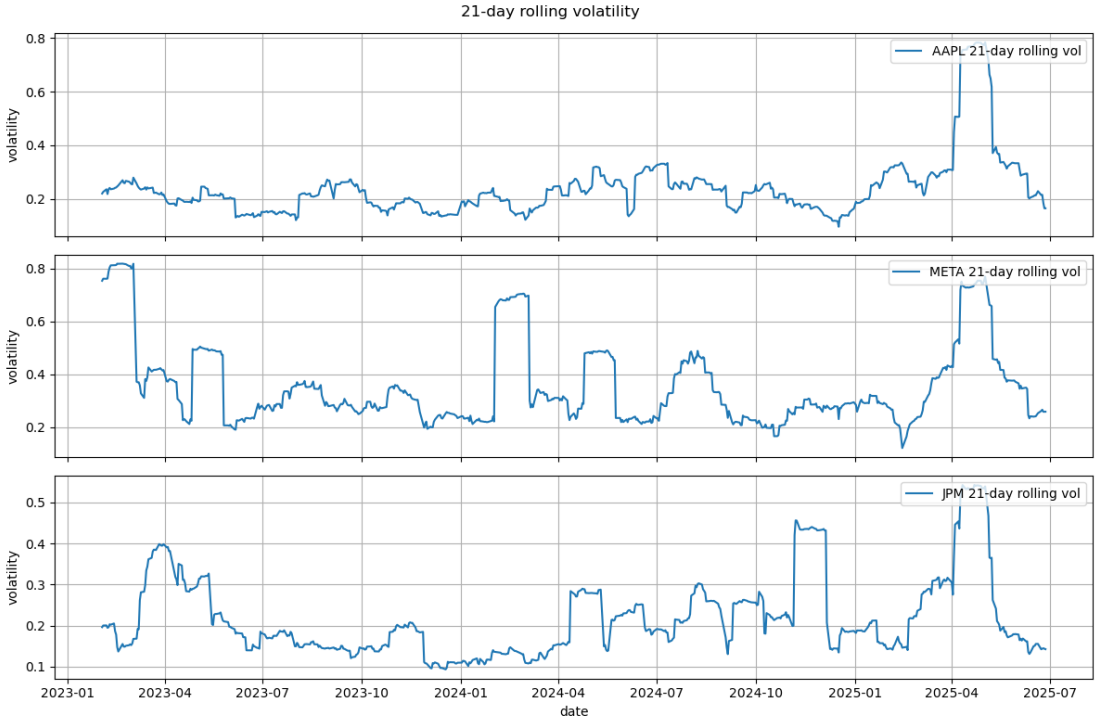

FIG. 2: Volatility trend

Comments. The 21-day rolling volatility plots (Fig. 2) show clear periods of calm and turbulence for all three stocks. AAPL and META display long stretches of low volatility followed by sharp increases, especially around 2025, confirming that large spikes seen earlier in their return series. META in particular shows the strongest swings, with several volatility jumps. JPM also experiences these volatility cracks, but they are generally smaller, and last less, which makes its volatility profile more stable compared to the two other stocks. Overall, the plots confirm that volatility is time-varying and clustered.

b) We now focus on correlation and autocorrelation between the assets returns. Especially we analyze $\text{Corr}(R_t^i, R_{t-h}^j)$ and $\text{Corr}(|R_t^i|, |R_{t-h}^j|)$ for $i, j \in$ **TICKERS** and $h \in \{0, 1, 2...25\}$, representing a lag.

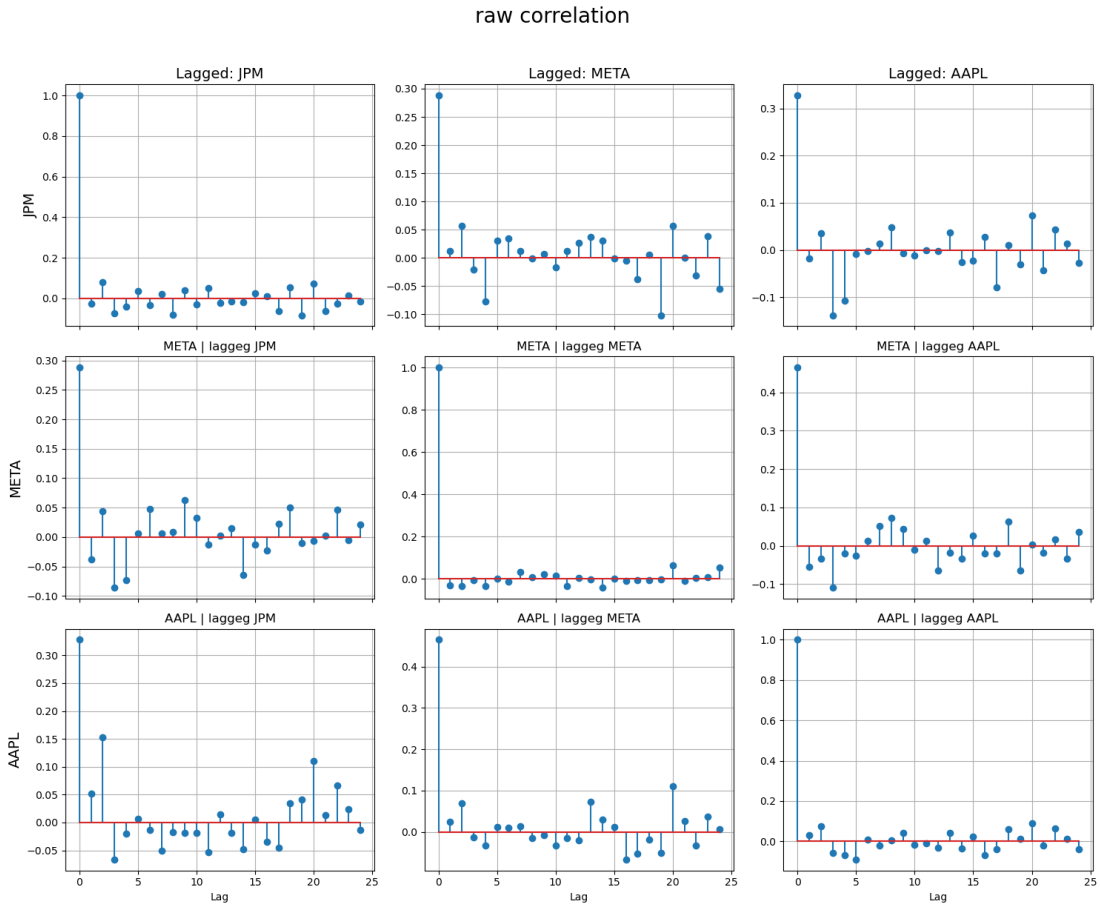

FIG. 3: Cross correlation for raw stocks

Comments. From the raw cross-correlation plots (Fig. 3) we see that auto-correlations and cross-correlations of raw returns are very close to zero at all lags. This shows that daily stock returns have very low linear dependence. The largest correlations occurs at lag 0. This reflects instantaneous co-movements driven by market-wide factors. There are a few small fluctuations, but nothing that looks consistent or different from random noise. The auto-correlations quickly drop near zero. Overall the plots suggest that, the three stocks do not strongly move together when looking at raw daily returns. But also that returns do not show strong patterns across time, which is why the lines remain close to zero at all lags.

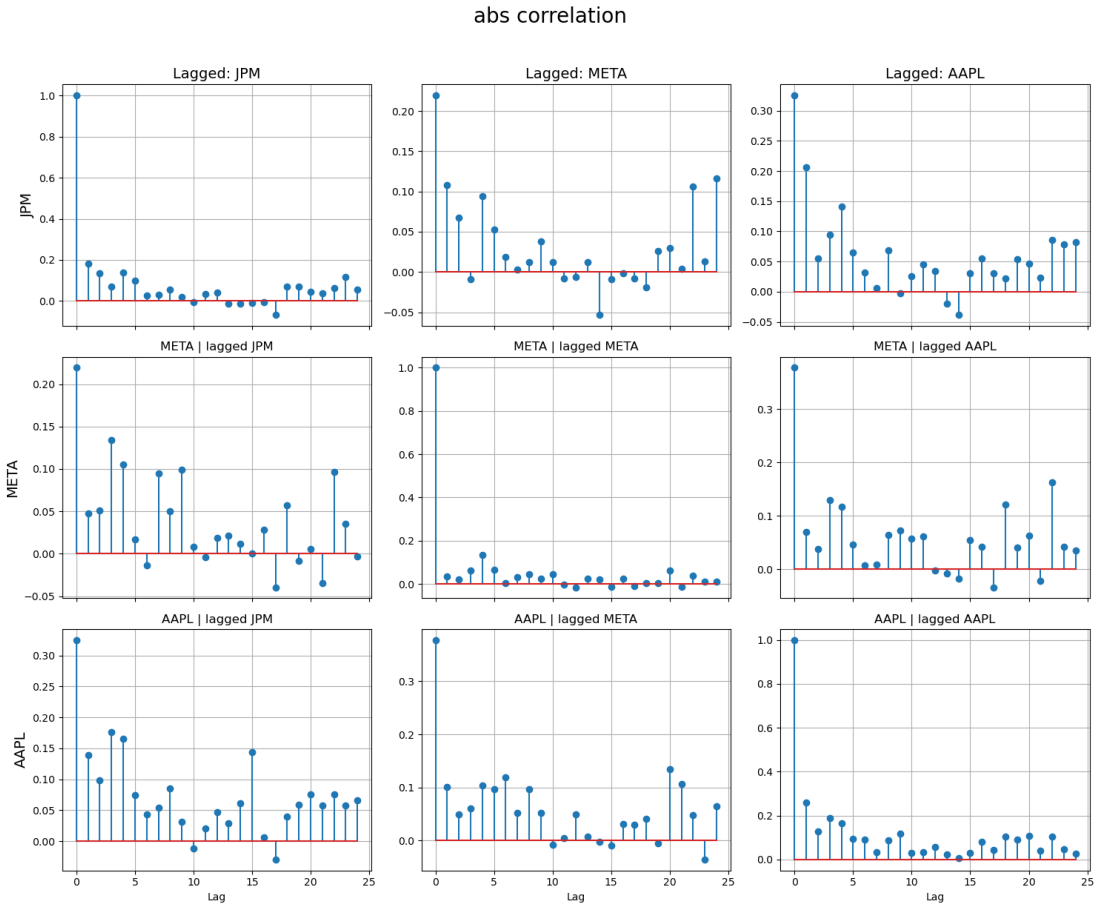

FIG. 4: Cross correlation for absolute-value stocks

Comments. When we look at the cross-correlations of the absolute returns (Fig. 4), the patterns look very different from the raw-return correlations. The first thing we notice is that the correlations are not as close to zero as for the raw returns. Several lags show positive and clear visible values, especially for first lags. This happens for all three assets in both auto-correlations and cross-correlations. More precisely, the auto-correlations are relatively strong within the first lags, and then slowly decrease, but they stay above zero for several lags. This means that when the absolute return is large on one day, it is likely to remain large for the next few days. The cross-correlations between different stocks are weaker, but much higher than in the raw-return case. This means that periods of high volatility tend to happen at the same time across the stocks. Taken together, the plots show that absolute returns are not random noise since they have structure, and the volatility of one stock is related to the volatility of the others, to some degree.

c) We want to assess normality of returns $\{R_t^i\}_t$, we start by plotting the QQ-plots against the normal distribution. What we do is computing the sample mean $\widehat{\mu}$ and

sample variance $\widehat{\sigma}^2$ of our returns, and plot the empirical quantiles of our returns versus the quantiles of a normal distribution $\mathcal{N}(\widehat{\mu}, \widehat{\sigma}^2)$. The empirical quantiles are just computed by ranking the returns, the i^{th} return is approximately $i + 0.5/(n+1)$ empirical quantile. If returns were normally distributed, the points would lie close to the red 45-degree line. We then plot the QQ-plot for the three different stocks in the following figure:

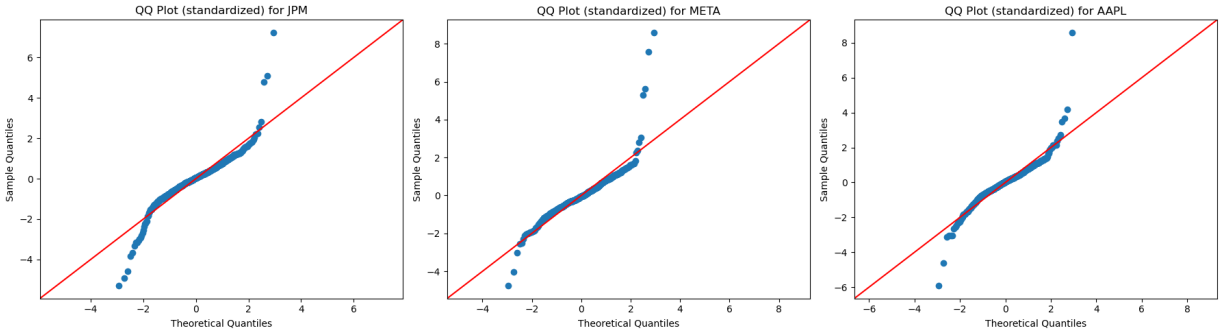

FIG. 5: QQ-plot

Comments. In all three QQ-plots (from Fig. 5), we observe clear deviations : the extreme left and right tails are far from the line; the points bend upward in the right tail and downward in the left tail; the central part is approximately linear, but the extremes show strong curvature. These patterns indicate heavy tails compared to the normal distribution, and do not motivate to use a gaussian representation.

Jarque-Bera Statistic: This statistic is used to test the null hypothesis of normality. We start by computing the sample skewness and kurtosis respectively:

$$b = \frac{\frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^3}{\left(\frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^2\right)^{3/2}} \qquad k = \frac{\frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^4}{\left(\frac{1}{n}\sum_{i=1}^{n}(X_i - \bar{X})^2\right)^{2}}$$

We can thus define the **Jarque-Bera statistic** as

$$T = \frac{1}{6}n\left(b^2 + \frac{1}{4}(k-3)^2\right)$$

which has an asymptotic χ^2_2-distribution under the null hypothesis of normality.

Asset	Test statistic	p-value	Result for H_0 (normality)
JPM	2203.51	0.0000	Reject
META	7608.76	0.0000	Reject
AAPL	3619.29	0.0000	Reject

TABLE 1: Normality test results for daily log-returns of the three assets.

Comments. The Jarque-Bera test strongly rejects the null hypothesis of normality for all three assets. Each test statistic is extremely large and all $p-$values are essentially zero, meaning the sample skewness and kurtosis are far from what would be expected under normal distribution. These results confirm what we observed in the QQ-plots : the distribution have clear deviation from normality, mainly in the tails. In particular, the very high kurtosis leads to heavy-tailed distribution, which explains the large deviations in the extreme quantiles. Hence, both graphical and statistical evidence point to the same conclusion : these daily log-returns are not normally distributed.

2. Single-window modeling: VaR, ES, and distributions.

We build an estimation window of $W = 252$ days, approximately one trading year. The goal of this section is to estimate the 1-day ahead Value at Risk (VaR) and Expected Shortfall (ES) based on different models. For each asset, we define the loss as $L_t^i = -R_t^i$

a) *Historical Simulation*: We suppose that the 1-day ahead loss follows the same distribution as our loss on the previous trading year. We thus compute a cdf based on our estimation window losses by sorting our losses $(\widetilde{L_k})_{k=1}^n$ $(\widehat{L}_1 \geqslant ... \geqslant \widehat{L}_n)$, and setting $\widehat{F}(\tilde{L}_k) = \frac{k}{n+1}$. The VaR at level α is thus the largest loss smaller than $\alpha\%$ of other losses. The ES is given by the mean of those losses bigger or equal to VaR.

$$\widehat{\mathrm{VaR}}_\alpha = \tilde{L}_{\lfloor n(1-\alpha)\rfloor+1} \qquad \widehat{\mathrm{ES}}_\alpha = \frac{1}{n(1-\alpha)} \sum_{i=1}^{\lfloor n(1-\alpha)\rfloor} \tilde{L}_i.$$

We then obtain the following two tables :

Asset	$\mathbf{VaR}_{0.95}$	$\mathbf{ES}_{0.95}$	$\mathbf{VaR}_{0.99}$	$\mathbf{ES}_{0.99}$
JPM	0.0183	0.0303	0.0376	0.0475
META	0.0273	0.0359	0.0431	0.0446
AAPL	0.0176	0.0268	0.0330	0.0407

TABLE 2: Historical VaR and ES

Now we plot the comparison between the true loss and the historical simulation :

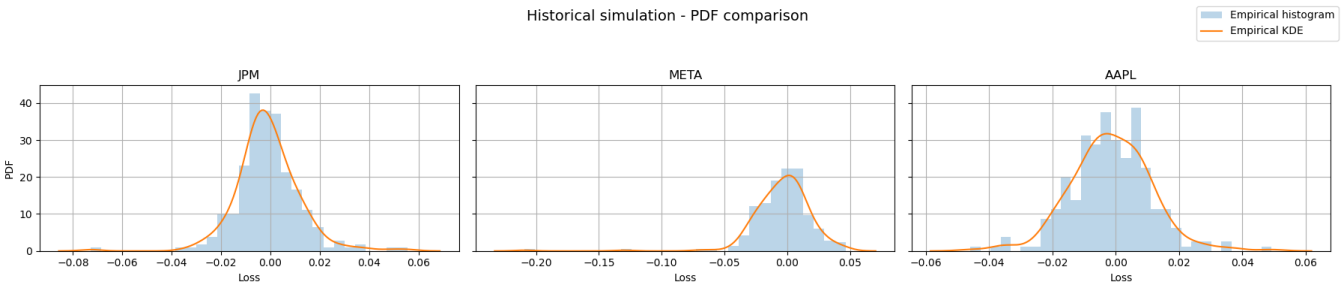

FIG. 6: Empirical Loss Distribution (Historical Simulation)

Comments. The empirical histogram and their corresponding KDE's provide a clear view of the loss distribution of each asset over the estimated window. We can see that JPM and AAPL exhibits more concentrated daily losses with most observations close to zero. In contrast, we see that the tails of the META loss distribution are thicker. This explains why META's historical VaR and ES exceed those of AAPL and JPM.

b) *Gaussian*: We suppose that the 1-day ahead loss follows a normal distribution $\mathcal{N}(\widehat{\mu}, \widehat{\sigma}^2)$, where $\widehat{\mu}$ and $\widehat{\sigma}^2$ are the sample mean and variance of our losses on the previous trading year. Under this assumption, we can use the gaussian distribution and definition of VaR and ES to compute them:

$$\begin{aligned}
\mathbb{P}(L \leqslant \widehat{\mathrm{VaR}}_\alpha) = \alpha &\iff \mathbb{P}\left(\frac{L-\widehat{\mu}}{\widehat{\sigma}} \leqslant \frac{\widehat{\mathrm{VaR}}_\alpha - \widehat{\mu}}{\widehat{\sigma}}\right) = \alpha \\
&\iff \widehat{\mathrm{VaR}}_\alpha = \widehat{\mu} + \widehat{\sigma}\Phi^{-1}(\alpha) \\
\widehat{\mathrm{ES}} = \mathbb{E}[L \mid L \geqslant \widehat{\mathrm{VaR}}_\alpha] &\iff \widehat{\mathrm{ES}} = \widehat{\mu} + \widehat{\sigma}\frac{\phi(\Phi^{-1}(\alpha))}{1-\alpha}
\end{aligned}$$

We then obtain the following Table:

Asset	$\mathbf{VaR}_{0.95}$	$\mathbf{ES}_{0.95}$	$\mathbf{VaR}_{0.99}$	$\mathbf{ES}_{0.99}$
JPM	0.0203	0.0257	0.0291	0.0336
META	0.0352	0.0452	0.0515	0.0596
AAPL	0.0194	0.0247	0.0281	0.0324

TABLE 3: Gaussian VaR and ES

We plot the comparison :

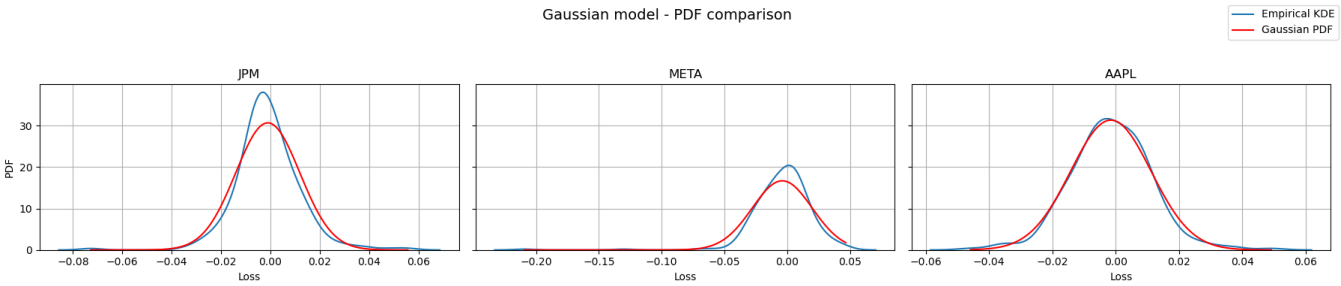

FIG. 7: Gaussian Model : Fitted PDF vs Empirical Losses

Comments. For JPM and META, we see that the peak of the Gaussian PDF is lower than that of the one of the Empirical KDE. This means that the Gaussian

model underestimates the number of observations close to zero. For AAPL however, the Gaussian PDF seems to follow the empirical KDE well. When we look closer to the tails, we see that AAPL and JPM Gaussian tails are thinner than Empirical ones. This implies that the Gaussian model assigns too little probability to large loss events. Because of this, the Gaussian ES for these two stocks is lower than the historical ES. We see the opposite pattern for META.

c) *Student-t*: We suppose that the 1-day ahead loss follows a student-t distribution $(\widehat{\nu}, \widehat{\mu}, \widehat{\sigma}^2)$, where the degree of freedom $\widehat{\nu}$, the mean $\widehat{\mu}$ and the variance $\widehat{\sigma}^2$ are estimated using maximum likelihood on the previous year. From the lecture notes, we have:

$$\widehat{\mathrm{VaR}}_\alpha = \widehat{\mu} + \widehat{\sigma} t_\nu^{-1}(\alpha)$$
$$\widehat{\mathrm{ES}}_\alpha = \widehat{\mu} + \widehat{\sigma} \frac{g_\nu(t_\nu^{-1}(\alpha))}{1-\alpha} \left(\frac{\nu + (t_\nu^{-1}(\alpha))^2}{\nu - 1} \right)$$

We obtain the following Table :

Asset	$\mathbf{VaR}_{0.95}$	$\mathbf{ES}_{0.95}$	$\mathbf{VaR}_{0.99}$	$\mathbf{ES}_{0.99}$
JPM	0.0179	0.0288	0.0340	0.0500
META	0.0300	0.0454	0.0534	0.0736
AAPL	0.0189	0.0264	0.0308	0.0391

TABLE 4: Student$-t$ VaR and ES

We plot the comparison :

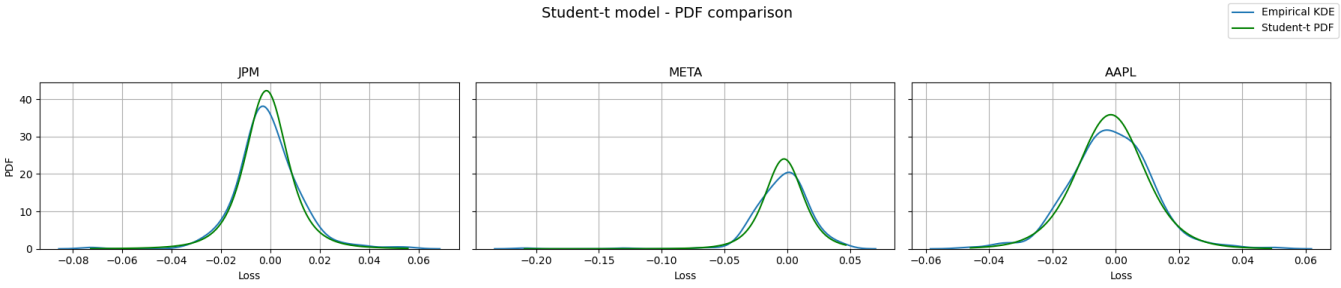

FIG. 8: Student$-t$ Model : Fitted PDF vs Empirical Losses

Comments. In the plots, the Student-t model seems to provide a better fit to the empirical losses distribution, especially in the tails. This is because its degrees-of-freedom parameter ν allows for heavier tails than the Gaussian model. This leads to VaR and ES that are generally closer to the historical benchmark than Gaussian's VaR and ES

d) *Conditional parametric* : In the previous approaches, we treated daily losses ad i.i.d, meaning the mean and volatility were constant through time. Financial time series however, often display more complicated dynamics : returns may exhibit short-term autocorrelation, and more importantly, volatility clustering. To account for these features, we use a conditional parametric model in which the *conditional mean* and the *conditional volatility* are modeled separately. More precisely, we estimate an AR(p) model for the conditional mean and a GARCH$(1,1)$ for the volatility.

- AR(p) *model for the conditional mean*: the loss process (L_t) is first modeled with an auto regressive model with p degrees:

$$L_t = \mu_t + \varepsilon_t, \quad \mu_t = \phi_0 + \sum_{i=1}^{p} \phi_i L_{t-i}$$

where p is chosen using plots from *Question 1*, Fig. 3 but also looking at the following PACF plots :

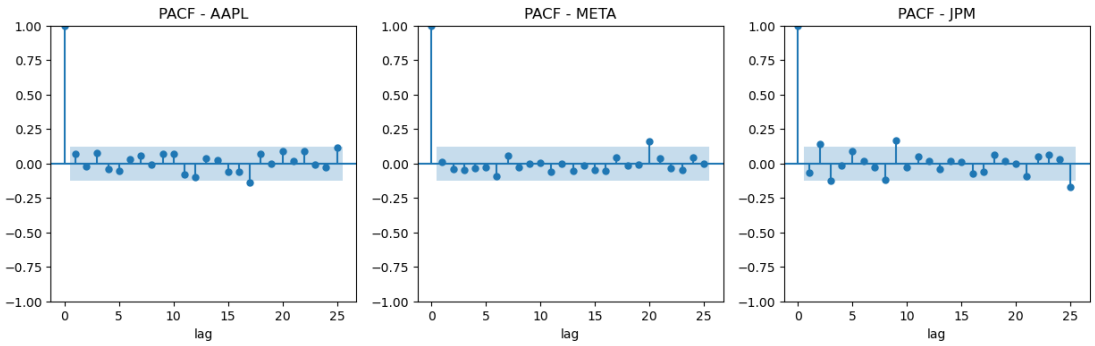

FIG. 9: Partial Autocorrelation function for three stocks

We then decide to chose $p = 0$ which means that there is no linear relationship between today's return and yesterday's. At first, we thought taking $p = 0$ was wrong, but when we did a regression, we observed that we failed to reject the hypothesis that the regression coefficient is zero, which is then consistent with taking $p = 0$. Given the estimation window of length $W = 252$, we fit the AR(0) model and compute the one-step-ahead forecast $\widehat{\mu}_{t+1}$. The AR residuals $\widehat{\varepsilon}_t$ are just given by $L_t - \widehat{\mu}$, where $\widehat{\mu}$ is the sample mean, which serve as input for the volatility model.

- GARCH$(1,1)$ *model for conditional volatility*: Volatility clustering is captured with a GARCH$(1,1)$ specification:

$$\sigma_t^2 = \alpha_0 + \alpha_1 \epsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

where $\epsilon_t^2 = Z_t^2 \sigma_t^2$ with $(Z_t)_{t\in\mathbb{Z}} \sim \text{SWN}(0,1)$. We then fit this model to the AR residuals, and get the one-step-ahead conditional volatility forecast, given by the square root of $\widehat{\sigma}_{t+1}^2$.

Now, given the conditional mean and volatility forecasts, the next-day loss is modelled as :

$$L_{t+1} = \widehat{\mu}_{t+1} + \widehat{\sigma}_{t+1} Z, \quad Z \sim \mathcal{N}(0,1).$$

Finally the VaR and ES at level α are given as for the Gaussian case, but using the forecasts rather than the simple estimators. We mention that we keep the fitted standardized residuals $z_t = \varepsilon_t / \sigma_t$ for the next model.

Asset	$\mathbf{VaR}_{0.95}$	$\mathbf{ES}_{0.95}$	$\mathbf{VaR}_{0.99}$	$\mathbf{ES}_{0.99}$
JPM	0.0145	0.0184	0.0209	0.0241
META	0.0314	0.0404	0.0461	0.0534
AAPL	0.0194	0.0247	0.0280	0.0323

TABLE 5: Conditional parametric (AR–GARCH) VaR and ES

We plot the comparison :

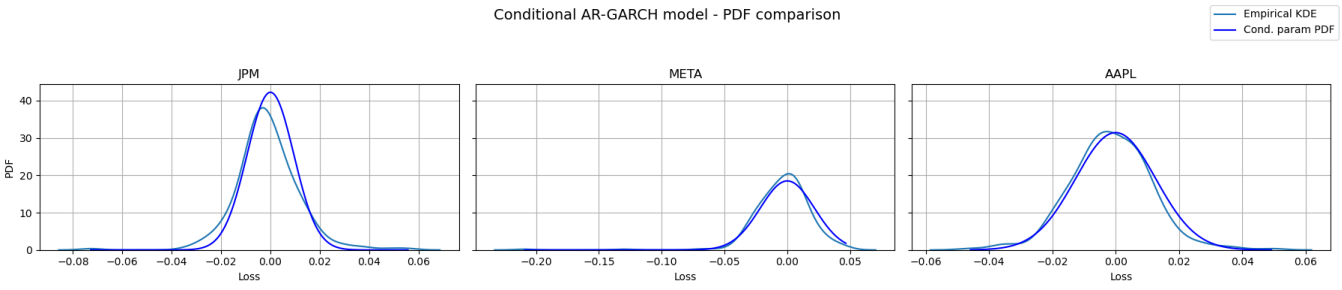

FIG. 10: Conditional Parametric Model : Fitted PDF vs Empirical Losses

Comments: Comparing the empirical losses PDF to the fitted with the conditional parametric model, we first notice the gaussian symmetry imposed by our model that does not align with the empirical losses. For JPM, the model has a thinner tail, underestimating the extreme losses probabilities. This is not he case for META nor AAPL. The advantage of this model is to take into account the trends in the time series, and thus be more efficient in predictions of future values. So comparing to the previous data is not the best way to evaluate the model.

e) *Filtered Historical Simulation* : the previous conditional parametric model assumes that the standardized innovations Z_t follow a Gaussian distribution. However this is not always the case, since we often see that financial returns exhibit heavier tails than the Gaussian law. Filtered Historical Simulation (FHS) addresses this by keeping this same AR-GARCH model for the conditional mean and volatility, but we replace the parametric assumption on the innovations with a non-parametric bootstrap.

- *Fit* $\mathrm{AR}(p) + \mathrm{GARCH}(1,1)$: we proceed as for model d), and then construct the standardized residuals. So once the GARCH model is fitted, we compute the standardized innovations $\widetilde{\varepsilon}_t = \widehat{\varepsilon}_t / \widehat{\sigma}_t$. Under the model, these should resemble an i.i.d sequence.
- *Bootstrap innovations*: instead of sampling $Z \sim \mathcal{N}(0,1)$, we draw

$$\widetilde{\varepsilon}_{W+1}^{(m)} \sim \{\widetilde{\varepsilon}_1, \ldots, \widetilde{\varepsilon}_W\} \text{ with replacement}, \quad m = 1, \ldots, M.$$

- *Generate simulated losses*: for each bootstrap draw, we build the one-step-ahead simulated loss :

$$L_{W+1}^{(m)} = \widehat{\mu}_{W+1} + \widehat{\sigma}_{W+1} \widetilde{\varepsilon}_{W+1}^{(m)}.$$

- *Compute* VaR *and* ES *from simulated distribution*: from the empirical distribution $(L_{W+1}^{(m)})_{m=1}^{M}$ we estimate the VaR and ES as for the model a).

Asset	$\mathbf{VaR}_{0.95}$	$\mathbf{ES}_{0.95}$	$\mathbf{VaR}_{0.99}$	$\mathbf{ES}_{0.99}$
JPM	0.0153	0.0213	0.0246	0.0335
META	0.0238	0.0306	0.0355	0.0384
AAPL	0.0172	0.0251	0.0287	0.0377

TABLE 6: Filtered Historical Simulation VaR and ES

We plot the comparison :

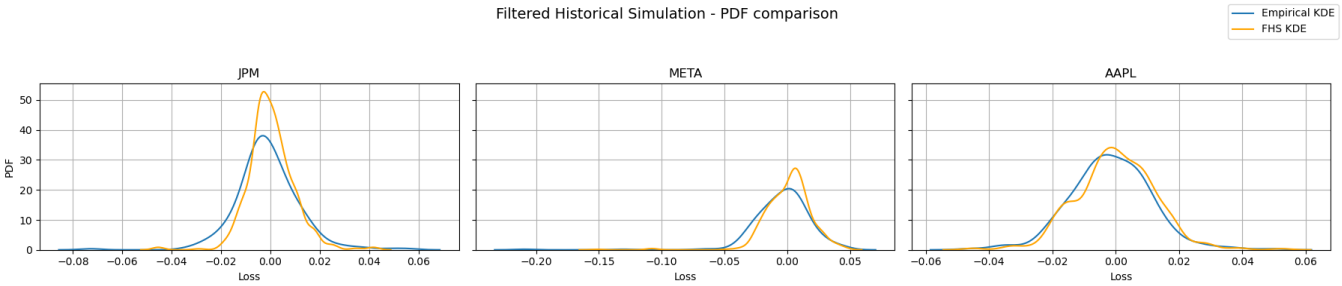

FIG. 11: Filtered Historical Simulation : Simulated vs Empirical PDF

Comments. We observe that the empirical PDF using FHS gives a distribution with thinner tails, has a higher mean and shows less pronounced shoulders. However for AAPL we get a correct fit, but that is not as smooth as for the two other fits.

3. Backtesting VaR and ES.

For this section, we use a rolling window of size $W = 252$ to produce 1-step ahead forecasts of VaR and ES at confidence intervals $\alpha \in \{0.95, 0.99\}$ for each assets and each method from *Question 2*. This means we fit our model on the estimation window $\{t - W + 1, ...t\}$, and produce forecasts for $t + 1$, advancing by one day each time until we reach the end of our sample. By doing so, we produce time series of $\widehat{\text{VaR}}_\alpha$ and $\widehat{\text{ES}}_\alpha$ from $t = W + 1$ to $t = T$, and want to asses the quality of these estimators by looking at the actual time series of realized losses. For all of the tests, we set the significant level at $a = 95\%$.

a) We start by assessing the quality of our $\widehat{\text{Var}}_\alpha$ estimators. The first test is the Kupiec **Proportion-Of-Failures (POF) test**, which tests if the observed frequency of $\widehat{\text{VaR}}_\alpha$ violations is consistent with the nominal level α. It is only based on the frequency of violations. We define :

$$I_t = \mathbf{1}\{L_t > \hat{q}_{t,\alpha}\}$$

The total number of violations N and the proportion of violations $\hat{p}$ are given by:

$$N = \sum_{t=1}^{T} I_t \quad , \quad \hat{p} = \frac{N}{T}$$

Under the null hypothesis and the definition of $\widehat{\text{VaR}}$, $\{I_t\}$ are i.i.d Bernoulli(p) with $p = 1 - \alpha$. We define the two likelihoods:

$$L_0 = (1-p)^{T-N} p^N$$
$$L_1 = (1-\hat{p})^{T-N} \hat{p}^N$$

We can thus construct the test statistic:

$$LR_{\text{POF}} = -2 \log \left(\frac{L_0}{L_1} \right)$$

As $T \to \infty$, $LR_{\text{POF}} \sim \chi^2_1$. We can thus compute a p-value as 1 - $\text{cdf}_{\chi^2_1}(LR_{\text{POF}})$.

Interpretation: if the $p-$value is smaller than a certain acceptance $5\% = (1 - a)$, we reject the null-hypothesis that the probability of violation is $1 - \alpha$, the model has biased coverage. Else we fail to reject the null hypothesis and the coverage is supposed to be adequate.

The **POF** test ignores clustering of violations in time. We thus use the **Christoffersen Independence and Conditional Coverage Tests**, which combines coverage and independence. We aim to model $\{I_t\}$ as a Markov chain with transition probabilities

$$\pi_{i,j} = \mathbb{P}(I_t = j | I_{t-1=i}), \quad i, j \in \{0, 1\}$$

We also introduce a transition count defined as :

$$N_{i,j} = \#\{t : I_{t-1} = i, I_t = j\}$$

such that:

$$\hat{\pi}_0 = \frac{N_{01}}{N_{00} + N_{01}} \qquad \hat{\pi}_1 = \frac{N_{11}}{N_{10} + N_{11}} \qquad \hat{p} = \frac{N_{01} + N_{11}}{N_{00} + N_{11} + N_{01} + N_{10}}$$

Given those number, we define two new likelihoods:

$$L_1 = (1 - \hat{\pi}_0)^{N_{00}} \hat{\pi}_0^{N_{01}} (1 - \hat{\pi}_1)^{N_{10}} \hat{\pi}_1^{N_{11}}$$
$$L_0 = (1 - \hat{p})^{N_{00}+N_{10}} \hat{p}^{N_{01}+N_{11}}$$

We can thus compute the **Independence test**:

$$LR_{\text{ind}} = -2 \log \left(\frac{L_0}{L_1} \right) \sim \chi_1^2$$

We can compute the p-value as before by taking the one minus the cdf of a χ_1^2 distribution.

Interpretation: We reject the test if the $p-$value is smaller than 5%. Test rejection means that the violations are not independent but clustered.

Conditional coverage test:

$$LR_{cc} = LR_{\text{POF}} + LR_{\text{ind}} \sim \chi_2^2$$

We once again compute the p-value by taking one minus the cdf of a χ_2^2 applied to the sample statistic.

Interpretation: The test tests if the sequence $\{I_t\}$ follows a Bernoulli trial with $p = (1 - \alpha)$, taking into account independence and frequency of violations. We reject the test if the p-value is smaller than 5%, meaning that the violations model is inadequate.

Model	α	LR	p-value	Decision
POF Test				
Historical	0.95	11.08	0.0009	Reject $H0$
	0.99	2.36	0.1248	Fail to reject $H0$
Gaussian	0.95	7.46	0.0063	Reject $H0$
	0.99	5.48	0.0193	Reject $H0$
Student-t	0.95	20.03	0.0000	Reject $H0$
	0.99	2.36	0.1248	Fail to reject $H0$
Conditional parametric	0.95	5.39	0.0203	Reject $H0$
	0.99	9.52	0.0020	Reject $H0$
FHS	0.95	7.46	0.0063	Reject $H0$
	0.99	0.42	0.5191	Fail to reject $H0$
Independence Test				
Historical	0.95	4.58	0.0323	Reject $H0$
	0.99	2.53	0.1114	Fail to reject $H0$
Gaussian	0.95	2.17	0.1411	Fail to reject $H0$
	0.99	1.62	0.2036	Fail to reject $H0$
Student-t	0.95	3.30	0.0694	Fail to reject $H0$
	0.99	2.53	0.1114	Fail to reject $H0$
Conditional parametric	0.95	0.04	0.8381	Fail to reject $H0$
	0.99	0.97	0.3238	Fail to reject $H0$
FHS	0.95	0.07	0.7930	Fail to reject $H0$
	0.99	3.90	0.0048	Reject $H0$
Conditional Coverage Test				
Historical	0.95	15.66	0.0004	Reject $H0$
	0.99	4.89	0.0867	Fail to reject $H0$
Gaussian	0.95	9.62	0.0081	Reject $H0$
	0.99	7.09	0.0288	Reject $H0$
Student-t	0.95	23.33	0.0000	Reject $H0$
	0.99	4.89	0.0867	Fail to reject $H0$
Conditional parametric	0.95	5.43	0.0662	Fail to reject $H0$
	0.99	10.49	0.0053	Reject $H0$
FHS	0.95	7.52	0.0232	Reject $H0$
	0.99	4.31	0.1156	Fail to reject $H0$

TABLE 7: Backtesting results for AAPL under all models: POF, independence, and conditional coverage tests.

AAPL VaR-*Backtesting Comments*:

- Concerning the **POF**, which focuses on the coverage of violations ($L_t > \widehat{\text{VaR}_\alpha}$), all the models reject H_0 for $\alpha = 0.95$ and only the *Conditional parametric* and the *Gaussian* models reject it for $\alpha = 0.99$. We can not reject the null hypothesis for the *Historical model*,the *Student-t model* and the *FHS* at $\alpha = 0.99$. This means that all the models tend to under-estimate VaR_α for $\alpha = 0.95$, leading to an underestimation of risk. For $\alpha = 0.99$, only the *Conditional Parametric* and the *Gaussian* models under-estimate the $\widehat{VaR}$, leading again to an underestimation of risk.
- Concerning the **Independence Test**, which focuses on the independence of violations, we observe two failures. It is for the *Historical* model at $\alpha = 0.95$ and *FHS* model at $\alpha = 0.99$. These rejections indicate that, for these models, violations tend to appear in clusters rather than being independently scattered through time. An explanation to this result concerning the *Historical* model is that when the stock has consecutive large losses, a single large value does not affect the historical estimation of $\widehat{\text{VaR}}$. The VaR only increases once the set of the $W \times (1 - \alpha)$ largest losses is modified. While for the other models such as *Gaussian* or *Student-t*, a single new outlier might affect the mean of the estimation window, and thus $\widehat{\text{VaR}}$.
- Concerning the **Conditional Coverage Test**, which combines both coverage and independence from the two previous test, we observe four failures at $\alpha = 0.95$: the *Historical*, *Gaussian*, *Student-t* and *FHS* models. The *Historical* model fails in both coverage and independence for $\alpha = 0.95$, it is logic that it doesn't pass this test either. The model underestimates the $\text{VaR}_{\alpha=0.95}$, and in a clustered not independent way. The *Gaussian*, *Student-t* and *FHS* models underestimate $\text{VaR}_{\alpha=0.95}$, and thus those three models are not adequate to build the risk measure $\widehat{\text{VaR}}_{\alpha=0.95}$. The *Conditional parametric* was not passing the POF test, but passes the independent, leading to passing also the Conditional Coverage test. This suggests that the independence of failures is strong for this model at $\alpha = 0.95$. But at $\alpha = 0.99$, we reject H_0 for the *Conditional Parametric* even though it was passing the independence test, suggesting that the coverage bias is strong.

Model	α	LR	p-value	Decision
POF Test				
Historical	0.95	1.0731	0.3002	Fail to reject $H0$
	0.99	2.3559	0.1248	Fail to reject $H0$
Gaussian	0.95	0.1314	0.7169	Fail to reject $H0$
	0.99	5.4771	0.0193	Reject $H0$
Student-t	0.95	1.5801	0.2087	Fail to reject $H0$
	0.99	0.4157	0.5191	Fail to reject $H0$
Conditional parametric	0.95	0.3719	0.5420	Fail to reject $H0$
	0.99	0.0239	0.8770	Fail to reject $H0$
FHS	0.95	0.1249	0.7238	Fail to reject $H0$
	0.99	0.4157	0.5191	Fail to reject $H0$
Independence Test				
Historical	0.95	0.2300	0.6315	Fail to reject $H0$
	0.99	0.2707	0.6028	Fail to reject $H0$
Gaussian	0.95	0.0611	0.8048	Fail to reject $H0$
	0.99	1.6165	0.2036	Fail to reject $H0$
Student-t	0.95	0.2601	0.6100	Fail to reject $H0$
	0.99	0.1374	0.7109	Fail to reject $H0$
Conditional parametric	0.95	1.4509	0.2284	Fail to reject $H0$
	0.99	0.0877	0.7672	Fail to reject $H0$
FHS	0.95	2.2935	0.1299	Fail to reject $H0$
	0.99	0.1374	0.7109	Fail to reject $H0$
Conditional Coverage Test				
Historical	0.95	1.3032	0.5212	Fail to reject $H0$
	0.99	2.6266	0.2689	Fail to reject $H0$
Gaussian	0.95	0.1925	0.9082	Fail to reject $H0$
	0.99	7.0936	0.0288	Reject $H0$
Student-t	0.95	1.8402	0.3985	Fail to reject $H0$
	0.99	0.5530	0.7584	Fail to reject $H0$
Conditional parametric	0.95	1.8228	0.4020	Fail to reject $H0$
	0.99	0.1116	0.9457	Fail to reject $H0$
FHS	0.95	2.4184	0.2984	Fail to reject $H0$
	0.99	0.5530	0.7584	Fail to reject $H0$

TABLE 8: Backtesting results for META: POF, independence and conditional coverage tests for different models.

META VaR-*Backtesting Comments*:

- Concerning the **POF**, none of the models are rejected at the 5% level for $\alpha = 0.95$. However, at $\alpha = 0.99$ the *Gaussian* model is rejected, which indicates a coverage bias: the number of VaR violations exceeds what the model predicts. This is consistent with the Gaussian distribution underestimating tail risk at extreme confidence levels. All other models pass the test, indicating that the number of VaR violations is consistent with what each model predicts.
- Concerning the **Independence Test**, again all models pass for both confidence intervals. The $p-$values are all well above 0.05, which indicates that violations do not exhibit clustering and seem independent in time.
- Concerning the **Conditional Coverage Test**, all models pass at $\alpha = 0.95$. At $\alpha = 0.99$ only the *Gaussian* model fails. Since this test evaluates jointly the number of exceedances and their independence, this result shows that the *Gaussian* model's failure is due to a coverage bias rather than clustered exceptions, as independence was not rejected. For all other models, the absence of rejection at both levels means that violations are not auto-correlated and that these models provide an adequate VaR evaluation over this period.

Model	α	LR	p-value	Decision
POF Test				
Historical	0.95	1.58	0.2087	Fail to reject $H0$
	0.99	3.79	0.0516	Fail to reject $H0$
Gaussian	0.95	0.01	0.9055	Fail to reject $H0$
	0.99	14.31	0.0002	Reject $H0$
Student-t	0.95	1.07	0.3002	Fail to reject $H0$
	0.99	11.83	0.0006	Reject $H0$
Conditional parametric	0.95	0.01	0.9055	Fail to reject $H0$
	0.99	9.52	0.002	Reject $H0$
FHS	0.95	1.58	0.2087	Fail to reject $H0$
	0.99	2.36	0.1248	Fail to reject $H0$
Independence Test				
Historical	0.95	3.22	0.0728	Fail to reject $H0$
	0.99	2.03	0.1538	Fail to reject $H0$
Gaussian	0.95	3.16	0.0753	Fail to reject $H0$
	0.99	0.53	0.4682	Fail to reject $H0$
Student-t	0.95	3.73	0.0534	Fail to reject $H0$
	0.99	0.73	0.3933	Fail to reject $H0$
Conditional parametric	0.95	3.16	0.0753	Fail to reject $H0$
	0.99	0.9736	0.3238	Fail to reject $H0$
FHS	0.95	1.24	0.2661	Fail to reject $H0$
	0.99	2.53	0.1114	Fail to reject $H0$
Conditional Coverage Test				
Historical	0.95	4.80	0.0908	Fail to reject $H0$
	0.99	5.82	0.0544	Fail to reject $H0$
Gaussian	0.95	3.18	0.2042	Fail to reject $H0$
	0.99	14.84	0.0006	Reject $H0$
Student-t	0.95	4.81	0.0904	Fail to reject $H0$
	0.99	12.56	0.0019	Reject $H0$
Conditional parametric	0.95	3.18	0.2042	Fail to reject $H0$
	0.99	10.49	0.0053	Reject $H0$
FHS	0.95	2.82	0.2445	Fail to reject $H0$
	0.99	4.89	0.0867	Fail to reject $H0$

TABLE 9: Backtesting results for JPM: POF, independence and conditional coverage tests for different models.

JPM VaR-*Backtesting Comments*:

- Concerning the **POF test**, results differ clearly across models. At $\alpha = 0.95$, none of the models are rejected. All models show $p-$values above 0.05, meaning they correctly capture the frequency of VaR violations. At $\alpha = 0.99$, the *Gaussian*, *Student–t* and the *Conditional parametric* models are rejected, while only the *Historical* and the *FHS* models pass.
- Concerning the **Independence Test**, all models pass at both $\alpha = 0.95$ and $\alpha = 0.99$. All $p-$values are well above the threshold, indicating that even when violations occur, they are not clustered in time.
- Concerning the **Conditional Coverage Test**, at $\alpha = 0.95$, all models fail to reject the null hypothesis. However, at $\alpha = 0.99$, *Gaussian*, *Student–t* and *Conditional Parametric* models are rejected, again due to an excessive number of breaches. The Historical and FHS models pass. These outcomes reinforce the idea that JPM's most extreme returns in this window show heavier tails than those implied by *Gaussian* or *Student–t* models.

b) **Acerbi and Székely Z_1 test**: This test is used to evaluate whether realized tail losses are consistent with the ES forecasts, given that the VaR forecasts at level α are correct. We denote by L_t the one step ahead loss, $(\widehat{q}_{t,\alpha}, \widehat{e}_{t,\alpha})$ the model forecasted VaR and ES. We also define the exception indicator $I_t = \mathbf{1}\{L_t > \widehat{q}_{t,\alpha}\}$, and the number of violations $N = \sum_{t=1}^{N} I_t$. We define the test statistic Z_1 as :

$$Z_1 = \frac{1}{N} \sum_{t=1}^{T} \frac{L_t I_t}{\widehat{e}_{t,\alpha}} - 1$$

if VaR and ES are correctly computed, the expectation of the loss, given that it is larger than the VaR, should be equal to the ES. So if both VaR and ES are correctly specified, we should have:

$$\mathbb{E}\left[\frac{L_t}{\widehat{e}_{t,\alpha}} - 1 \mid I_t = 1 \right] = 0.$$

Under the null hypothesis, Z_1 should be close to zero. A problem that needs to be solved is that we don't have a sample distribution for Z_1, but a single value. To solve this issue, simulation is needed to obtain a reference distribution of Z_1. The idea is to use the distribution used to compute VaR and ES, but to generate M artificial losses $\{L_t^{(m)}\}_{m=1}^{M}$ at each forecast date t. By doing so, we have M simulated paths of losses, on which we can compute $I_t^{(m)}$ and $N^{(m)}$, as well as the Z_1 statistics, discarding the one with $N = 0$:

$$Z_1^{(m)} = \frac{1}{N^{(m)}} \sum_{t=1}^{T} \frac{L_t^{(m)} I_t^{(m)}}{\widehat{e}_{t,\alpha}} - 1$$

We then have a sample $\{Z_1^{(m)}\}$ to approximate the distribution of Z_1 under the model. This allows us to compute a p-value given by:

$$p = \frac{1}{M} \sum_{i=1}^{M} \mathbf{1}\{Z_1^{(m)} \geqslant Z_1\}$$

We reject the null Hypothesis if the $p - value$ is below the chosen significance level (in our case 5%. This would mean Z_1 is unusually large compared to its simulated null distribution, and so the realized tail losses are heavier than predicted by the model and its ES.

Model	$\boldsymbol{\alpha}$	$\boldsymbol{Z_1}$	***p*-value**	**Decision**
		Z1 ES **backtest**		
Historical	0.95	0.0438	0.2440	Fail to reject H_0
	0.99	0.1802	0.0152	Reject H_0
Gaussian	0.95	0.1932	0.0000	Reject H_0
	0.99	0.3514	0.0000	Reject H_0
Student-t	0.95	-0.0149	0.4780	Fail to reject H_0
	0.99	0.0429	0.3261	Fail to reject H_0
ARCH–GARCH	0.95	0.2292	0.0010	Reject H_0
	0.99	0.2618	0.0062	Reject H_0
FHS	0.95	0.0883	0.2230	Fail to reject H_0
	0.99	0.3258	0.0302	Reject H_0

TABLE 10: Z1 ES backtest results for AAPL under different models.

AAPL ES *backtest comments*:

- $\alpha = 0.95$: We reject null hypothesis for the *Gaussian* and *Conditional-Parametric* models. For the two models, Z_1 is unusually large compared to the simulated null distributions, which means that the realized tail losses are heavier than predicted by the two models. The models under-estimate $\text{ES}_{\alpha=0.95}$, and thus under-estimate the risk of AAPL returns. This can be explained by the fact that those two models suppose gaussian distribution of the losses, and thus even if mean and variance of the loss distribution are correctly estimated, the tails probability are underestimated and so the ES.
- $\alpha = 0.99$: We reject the null hypothesis for the *Historical*, *Gaussian*, *Conditional-Parametric* and *FHS* models. The models under-estimate $\text{ES}_{\alpha=0.99}$, and thus under-estimate the risk of AAPL returns. Concerning the *Historical* model, At α= 0.99, we calculate the mean of the worst 3 observations as historical ES. If one new "bad day" enters the window, the ES estimation jumps. This instability makes it hard to consistently pass backtests at such a high α. The most adequate

model to estimate the expected shortfall is thus the *Student-t* model, which fails to reject H_0 for the two values of α, and thus does not under-estimate the tail losses of AAPL.

Model	$\boldsymbol{\alpha}$	**Z1**	***p*-value**	**Decision**
Historical	0.95	0.0896	0.1700	Fail to reject H_0
	0.99	0.1775	0.1312	Fail to reject H_0
Gaussian	0.95	0.2702	0.0000	Reject H_0
	0.99	0.1741	0.0237	Reject H_0
Student-t	0.95	0.0625	0.2130	Fail to reject H_0
	0.99	0.0871	0.2222	Fail to reject H_0
ARCH-GARCH	0.95	0.1788	0.0010	Reject H_0
	0.99	0.4384	0.0000	Reject H_0
FHS	0.95	0.0842	0.2450	Fail to reject H_0
	0.99	0.2517	0.2193	Fail to reject H_0

TABLE 11: Z1 ES backtesting results for META across all five models.

META ES *backtest comments*:

- $\alpha = 0.95$: We reject null hypothesis for the *Gaussian* and *Conditional-Parametric* models. For the two models, Z_1 is unusually large compared to the simulated null distributions, which means that the realized tail losses are heavier than predicted by the two models. The models under-estimate $\text{ES}_{\alpha=0.95}$, and thus under-estimate the risk of META returns. This one again might come from a under estimation of tails by a gaussian approximation.
- $\alpha = 0.99$: We reject the null hypothesis for *Gaussian* and *Conditional-Parametric* models again. The models under-estimate $\text{ES}_{\alpha=0.99}$, and thus under-estimate the risk of META returns. The most adequate models to estimate the expected shortfall are thus the *Historical*, *Student-t*, *FHS* models, which both fail to reject H_0 for the two values of α, and thus do not under-estimate the tail losses of META.

Model	α	Z_1	p-value	Decision
Historical	0.95	0.1545	0.0750	Fail to reject H_0
	0.99	0.1654	0.0818	Fail to reject H_0
Gaussian	0.95	0.5935	0.0000	Reject H_0
	0.99	0.4447	0.0000	Reject H_0
Student-t	0.95	0.3516	0.0210	Reject H_0
	0.99	0.0828	0.2622	Fail to reject H_0
ARCH-GARCH	0.95	0.6031	0.0000	Reject H_0
	0.99	0.3963	0.0021	Reject H_0
FHS	0.95	0.1289	0.1890	Fail to reject H_0
	0.99	0.0883	0.4937	Fail to reject H_0

TABLE 12: Z1 ES backtesting results for JPM across different models.

JPM ES *backtest comments*:

- $\alpha = 0.95$: We reject null hypothesis for the *Gaussian*, *Student-t* and *Conditional-Parametric* models. The *Student-t* model under estimates the ES for the first time with the JPM stock. This result is interesting because the Student-t distribution is specifically designed to handle fat tails (kurtosis). The fact that it still failed suggests that modeling the shape of the distribution alone is not enough, we also need to model the dynamics. The failure is likely due to volatility clustering: since the model is static over the estimation window, it reacts too slowly to sudden bursts of volatility, which are common in the banking sector. Additionally, the symmetric nature of the standard Student-t distribution might fail to capture the negative skewness of JPM returns, where losses are often larger than gains.
- $\alpha = 0.99$: We reject the null hypothesis for *Gaussian* and *Conditional-Parametric* models again. The models under-estimate $\text{ES}_{\alpha=0.99}$, and thus under-estimate the risk of JPM returns. The most adequate models to estimate the expected shortfall are thus the *Historical*, ans *FHS* models, which both fail to reject H_0 for the two values of α, and thus do not under-estimate the tail losses of JPM.

4. Copula fitting.

a) For this part, we work on a single estimation window of length $W = 252$ for the log-returns of JPM, META and AAPL. For each asset i and each date $t = 1, \dots, W$, we construct the pseudo-observation

$$U_{t,i} = \frac{\text{rank}(R_{t,i})}{W+1}$$

where $\text{rank}(R_{t,i})$ is the rank of $R_{t,i}$ among $\{R_{1,i}, \dots, R_{W,i}\}$. The vectors $(U_{t,\text{JPM}}, U_{t,\text{META}}, U_{t,\text{AAPL}})$ are therefore empirical observations from the coupla of the tree assets.

We first plot the dependence between asset pars in the raw returns space (i.e. plot of $(R_{t,i}, R_{t,j})$) and obtain the following :

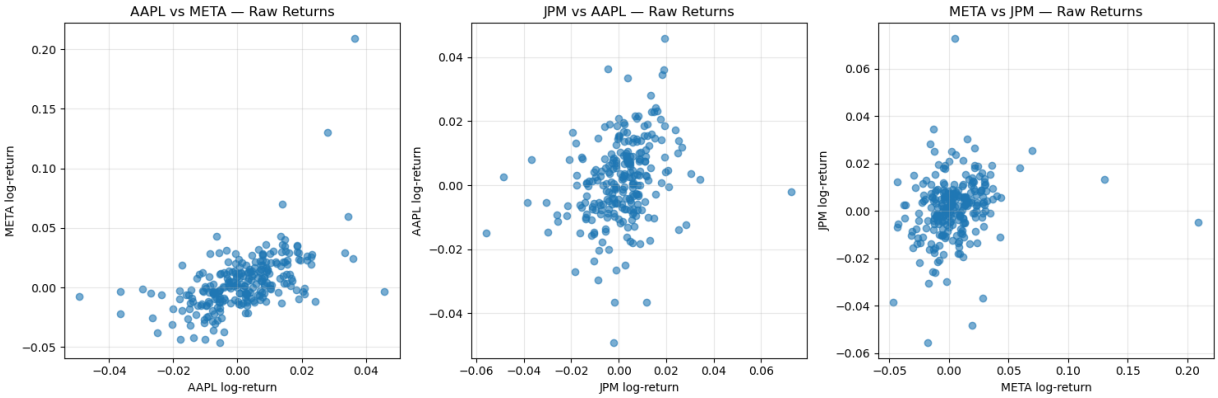

FIG. 12: Scatterplot of (R_i, R_j)

Comments.

- AAPL vs META : this scatter shows a moderate positive dependence, with an upward slope. A few joint large positive returns suggest slightly upper-tail co-movement, while negative extremes are rarer. Overall, the cloud remains symmetric around the origin, with only light asymmetry.
- JPM vs AAPL : this pair exhibit weaker positive dependence, with most points clustered around zero. There is very joint tail activity, indicating almost no tail dependence. The pattern is fairly symmetric, without clear non-linear dependence.
- META vs JPM : this scatter displays a moderate positive dependence, stronger than JPM-AAPL, but similar to AAPL-META. Some upper-right points indicate slightly stronger upper-tail co-movement, though the effect is weak. Symmetry is roughly preserved, with no meaningful lower-tail dependence.

Overall, the raw-return scatterplots show weak to moderate positive dependence across all asset pairs, with little evidence of strong tail dependence. The patterns remain fairly symmetric and roughly elliptical, suggesting that Gaussian or Student$-t$ copulas are suitable first choices, while strongly asymmetric or tail-dependent copulas are less justified at this stage.

We then plot the same scatterplot but in the pseudo-observations space and get:

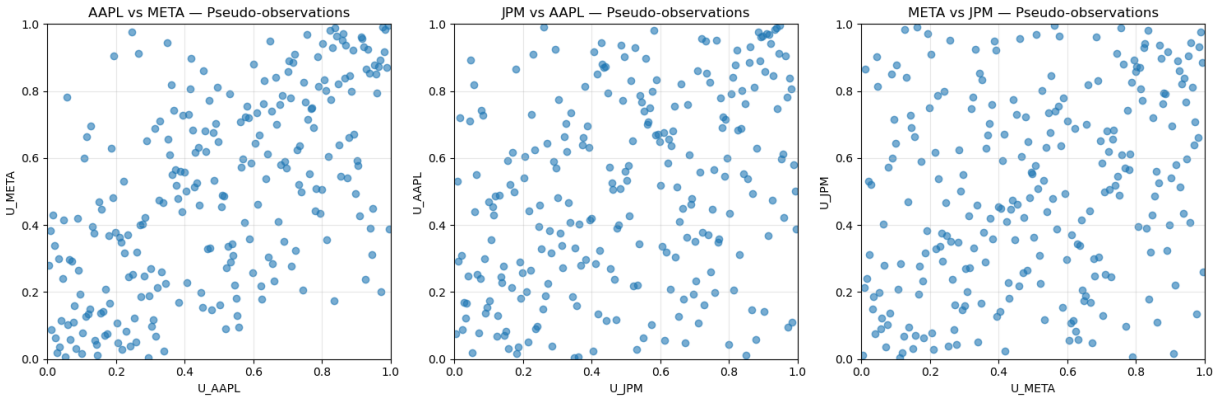

FIG. 13: Scatterplot of $(U_{i,t}, U_{j,t})$

Comments.

- Across all thee pairs, the points are spread quite uniformly in the unit square, with only a mild concentration across the diagonal. This indicates weak to moderate overall dependence, consistent with what we saw in the previous return scatterplots.
- The clouds look roughly symmetric, with no clear compression towards only one of the corners. This suggest no strong asymmetric dependence.
- We do not observe many points clustering near the bottom-left corner (so weak lower-tail dependence) nor the top-right corner (so weak upper-tail dependence). Instead, the corners remain thin, so the data do not indicate significant tail co-movements.
- Given the weak overall dependence, the approximate symmetry and the very little tail concentration, as we have seen before, the dependence structure is compatible with elliptical copulas such as the Gaussian copula (no tail dependence) or the Student$-t$ copula (symmetric tail dependence).

b) To fit the copulas, we proceed in two steps :

- *Build pseudo-observations* : we take the $W = 252$ log-returns of each asset in the estimation window and transform them as in part a). This gives us a $W \times 3$ matrix of pseudo-observations.
- *Gaussian copula* : for the Gaussian copula, we estimate the correlation matrix $\hat{R}^{(G)}$ by maximum likelihood : we choose the 3×3 correlation matrix that makes the observed pseudo-observations most likely under the Gaussian copula density. This is achieve using the `.fit` from the copula library on the matrix of $(U_{1,t}, U_{2,t}, U_{3,t})$, which returns the correlation matrix.
- *Student$-t$ copula*: for the Student$-t$ copula, we estimate both degrees of freedom $\hat{\nu}$ and the correlation matrix $\hat{R}^{(T)}$. Again we use the maximum likelihood : we

pick ν and R that maximize the copula log-likelihood of pseudo-observations. In the code it's done again using `.fit` function of the Student$-t$ copula, which outputs $\hat{\nu}$ and $\hat{R}^{(T)}$.

- *Gaussian copula*: for the Gaussian copula, the parameter vector is just the 3×3 correlation matrix $\widehat{R}^{(G)}$. In our estimates, the pairwise correlations are approximately :

$$\rho^{(G)}_{\text{JPM,META}} \approx 0.2870, \quad \rho^{(G)}_{\text{JPM,AAPL}} \approx 0.3428, \quad \rho^{(G)}_{\text{META,AAPL}} \approx 0.5979.$$

- *Student$-t$ copula* : for the Student$-t$ copula we have two type of parameters. First, the correlation matrix $\widehat{R}^{(T)}$, but also the degrees-of-freedom parameter $\widehat{\nu}$. The fitted model yields :

$$\widehat{\nu} \approx 13, \quad \rho^{(T)}_{\text{JPM,META}} \approx 0.3021, \quad \rho^{(T)}_{\text{JPM,AAPL}} \approx 0.3494, \quad \rho^{(T)}_{\text{META,AAPL}} \approx 0.5995.$$

c) Once the Gaussian and Student$-t$ copulas are fitted on the pseudo-observations, we use them to generate synthetic data.

- *Simulate uniform samples from fitted copulas* : for each copula $C_{\hat{\theta}}$ we draw

$$(U_1^{(m)}, U_2^{(m)}, U_3^{(m)}) \sim C_{\hat{\theta}}, \quad m = 1, \ldots, T.$$

  More precisely, we used the `copulae` library to generate samples from both fitted copulas.

- *Recover marginal distributions via inverse empirical CDF* : for each asset i, we approximate its marginal distribution using its historical returns in the estimation window (so the last $W = 252$ observations). So for each asset i, we take the $W = 252$ returns in the estimation window, sort them, and construct the empirical CDF. Then we compute the empirical quantile using the inverse CDF $\hat{F}_i^{-1}(u)$ using only those 252 observations. And for each simulated uniform value $U_i^{(m)}$ from the Gaussian or Student$-t$ copula, we transform it via $\tilde{R}_i^{(m)} = \hat{F}_i^{-1}(U_i^{(m)})$ obtaining a simulated return. We note that it's essential to apply the inverse CDF based on the W-window only, because the copula itself is fitted using exactly those same W observations.

We first plot the comparison of the marginal density :

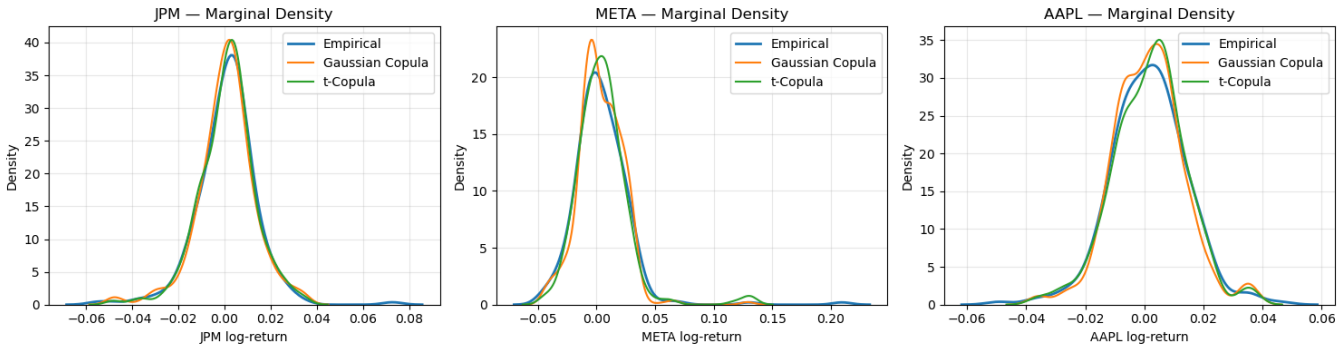

FIG. 14: Marginal densities

Comments.

- JPM : the simulated Gaussian and Student$-t$ copula returns match the empirical density well in the center of the distribution. The Gaussian copula slightly underestimates tail thickness, while the Student$-t$ copula better captures the heavier tails observed in the empirical data.
- META : META exhibits the strongest tail behavior among the three assets. The empirical density shows clear heavy tails, which neither copula fully reproduces after the inverse CDF transformation. The Gaussian copula-based simulations display noticeably thinner tails, while the Student-t copula captures tail thickness more accurately, although it still underestimates the most extreme observations. This is expected, as both copula simulations rely on only 252 empirical points to reconstruct the marginal, limiting their ability to replicate extreme values.
- AAPL : for AAPL, both copulas reproduce the empirical marginal distribution well. The three curves almost overlap perfectly across the center and both tails of the distribution, with some deviations. Unlike META, AAPL does not exhibit strong tail behavior, so the heavy–tailed $t-$copula offers no visible advantage over the Gaussian copula.

And now we plot the original and simulated returns in the following three plots:

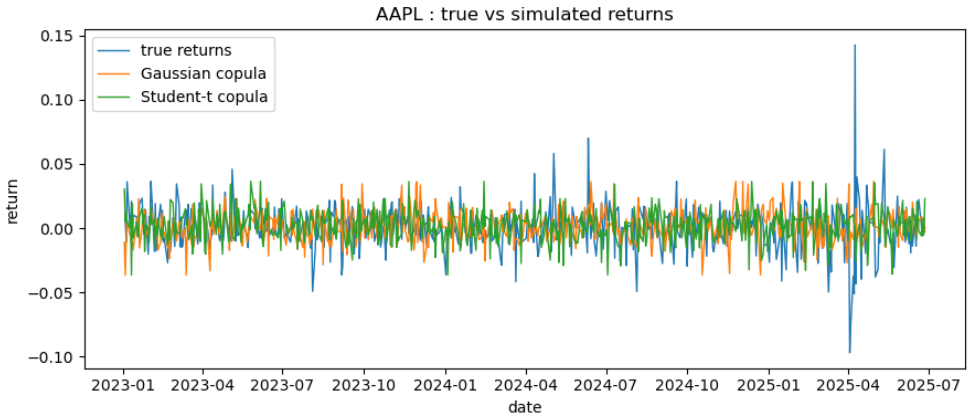

FIG. 15: AAPL : True returns vs Simulated Copula returns

Comments. The synthetic return series generated from the Gaussian and Student$-t$ copulas reproduce the overall variability and time-series structure of the AAPL returns, remaining centered around similar levels. The main difference lies in the extremes : the true returns exhibit several sharp jumps that neither copula fully replicates, as expected given the smoothness from the empirical marginals. Between the two, the Student$-t$ copula produces more slightly more dispersed simulated paths.

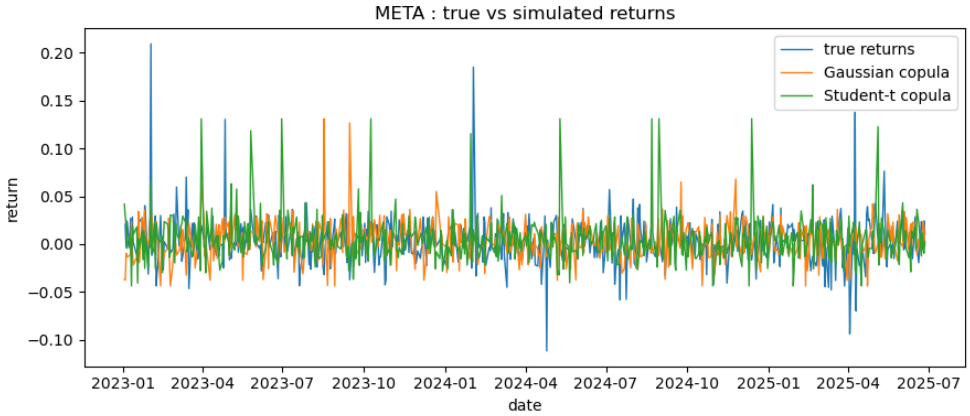

FIG. 16: META : True returns vs Simulated Copula returns

Comments. For META, both copula simulations capture the general volatility level and fluctuations of the true returns. However neither model fully reproduces the magnitude and frequency of the largest positive and negative jumps visible in the empirical series. The Gaussian copula tends to produce the smoothest trajectories with fewer extremes, while the Student$-t$ copula generates slightly more dispersion, yet still underestimates the sharpest spikes. Overall, the true META returns exhibit more tail behavior than what either copula simulation is able to replicate.

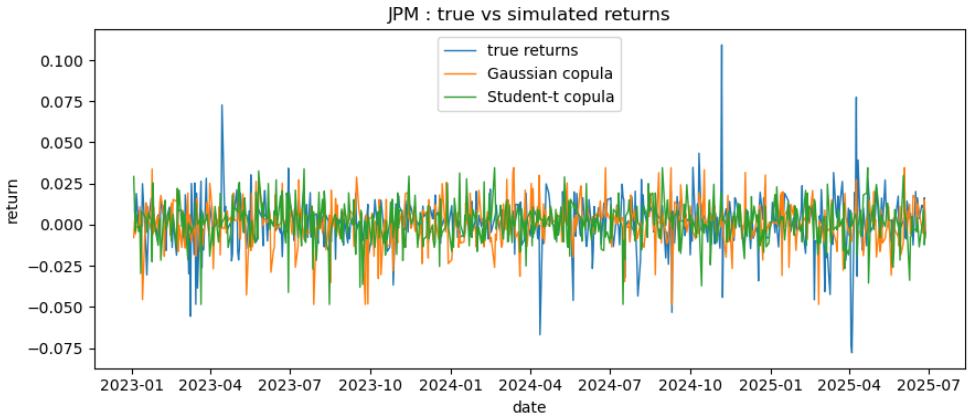

FIG. 17: JPM : True returns vs Simulated Copula returns

Comments. For JPM, both copula simulations reproduce the overall volatility and fluctuations of the empirical results. As in the other assets, extreme spikes in the true series occasionally exceed those generated by the simulations, showing the smoothness from the copula samplings. The Gaussian and Student$-t$ paths are visually very similar, with the second one producing slightly heavier shocks. Overall both copulas succeed at capturing the variability of JPM returns

5. Backtesting portfolio VaR and ES.

We now build an equally weighted portfolio built with the three assets. We model the returns of our portfolio by:

$$R_t^{\text{pf}} = \frac{1}{3}(R_t^{AAPL} + R_t^{META} + R_t^{JPM}), \quad L_t^{\text{pf}} = -R_t^{\text{pf}}.$$

The objective is to compute the 1-day-ahead VaR and ES forecasts.

a) *Univariate models*: We start by building the portfolio returns time series, and treat it as we treated the three assets in *Question 3*. At each time t, the model yields a 1-day-ahead $\text{VaR}_{\alpha,t+1}$ and $\text{ES}_{\alpha,t+1}$ for $\alpha = 0.95$ and 0.99. We collect these forecasts together with the realized loss L_{t+1}, producing a full OOS predictions. For model d) applied to the portfolio, we also take a look at the PACF plot to choose the value for p :

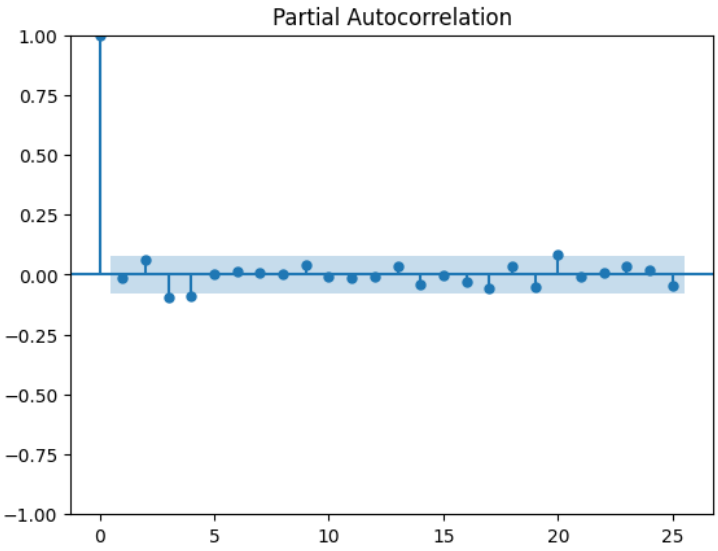

FIG. 18: PACF Plot for portfolio

Comments. We clearly see as for *Question 3* that we choose $p = 0$.

We then apply the Kupiec POF, Christofferesen independence, and a conditional coverage tests for VaR, as well as the Acerbi-Székely Z_1 test for ES.

b) *Copulas*: to incorporate dependence among the three assets, we use the copulas fitted in *Question 4*. For each time t, we proceed as follows:

- *Form the rolling window of asset returns*: we define the vector

$$(R^{\text{JPM}}_{t-W+1:t}, R^{\text{META}}_{t-W+1:t}, R^{\text{AAPL}}_{t-W+1:t})$$

and convert each series into pseudo-observations as we did before, i,e, :

$$U_{i,t} = \frac{\text{rank}(R^i_t)}{W+1}, \quad i \in \{\text{JPM}, \text{META}, \text{AAPL}\}.$$

- *Refit the copula on the window* : we fit the Gaussian copula with correlation matrix Σ_t, and the Student$-t$ copula with (ν_t, Σ_t).
- *Simulate portfolio dependence using the fitted copula*: for each day t, we draw $N = 1000$ samples $U_t^{(m)}$ from the fitted copula (Gaussian or Student$-t$). These samples represent synthetic shocks for the three assets.
- *Recover the marginal distribution using the inverse empirical CDF*: For each asset i, we transform each simulated uniform sample into a return via:

$$\tilde{R}^{(m)}_{i,t} = \hat{F}^{-1}_{i,t}(U^{(m)}_{i,t}),$$

where $\hat{F}^{-1}_{i,t}$ is the empirical inverse CDF built from the W observations.

- *Estimate portfolio* VaR/ES *from simulated returns*: we compute portfolio returns and estimate $\text{VaR}_{\alpha,t}$ and $\text{ES}_{\alpha,t}$.

c) *Backtesting and comparison*:

Model	**POF**	**Independence**	**Cond. Coverage**	**Z1 ES**
Historical	Fail to reject H_0	Fail to reject H_0	Fail to reject H_0	Reject H_0
Gaussian	Fail to reject H_0	Fail to reject H_0	Fail to reject H_0	Reject H_0
Student-t	Fail to reject H_0	Fail to reject H_0	Fail to reject H_0	Reject H_0
Conditional parametric	Fail to reject H_0	Fail to reject H_0	Fail to reject H_0	Reject H_0
FHS	Fail to reject H_0	Fail to reject H_0	Fail to reject H_0	Reject H_0
Gaussian Copula PF	Reject H_0	Fail to reject H_0	Reject H_0	Reject H_0
Student-t Copula PF	Fail to reject H_0	Fail to reject H_0	Fail to reject H_0	Fail to reject H_0

TABLE 13: Backtesting decisions for all models under $\alpha = 0.95$.

Comments. For $\alpha = 0.95$, all univariate models have the same decisions for the VaR tests : all models fail to reject H_0 in the POF, Independence and Conditional Coverage tests. This means that for this α, each model produces a number of violations close to what's expected, without clustering. However, all univariate models reject H_0 under the Z_1 backtest, indicating an underestimation of the shortfall at the 95% level. When looking at copula-based portfolio models, we observe a different conclusion. The *Gaussian-copula* portfolio fails the POF and Conditional Coverage tests, suggesting that its simulated portfolio returns generate too many violations. However, the *Student−t-copula* portfolio passes all VaR tests, behaving similarly to the univariate models. We see that the *Student-t-copula* is the only one that pass the Z_1 test. This means that it is the only one that does not underestimate large portfolio losses.

Note: while running the Z_1 ES Test for the Student$-t$ copula-based portfolio, we found that its $p-$value was very close to the threshold. And due to numerical instability, we sometimes obtained the other decision for this test, so we should be careful.

Model	**POF**	**Independence**	**Cond. Coverage**	**Z1 ES**
Historical	Reject H_0	Fail to reject H_0	Reject H_0	Reject H_0
Gaussian	Reject H_0	Fail to reject H_0	Reject H_0	Reject H_0
Student-t	Reject H_0	Fail to reject H_0	Reject H_0	Fail to reject H_0
Conditional parametric	Reject H_0	Fail to reject H_0	Reject H_0	Reject H_0
FHS	Reject H_0	Fail to reject H_0	Reject H_0	Reject H_0
Gaussian Copula PF	Reject H_0	Fail to reject H_0	Reject H_0	Reject H_0
Student-t Copula PF	Reject H_0	Fail to reject H_0	Reject H_0	Fail to reject H_0

TABLE 14: Backtesting decisions for all models under $\alpha = 0.99$.

Comments. For $\alpha = 0.99$ we reject H_0 for the proportion of failures for all models, as well as the conditional coverage test. But we again fail to reject H_0 for the independence test. This means that most models compute $\widehat{\text{VaR}}_{\alpha=0.99}$ that are inadequate in the sense that coverage of failures ($L > \text{VaR}$) is not $(1 - \alpha)$. Those failures are, on the other hand, independent. Concerning the Back-testing on the ES, the *Student-t*, univariate and copula-based models both fail to reject H_0. This means that are appropriately estimating the extreme losses of the portfolio, while all of the other models under-estimate the risk.

Comments on dependence. For $\alpha = 0.95$, the Gaussian copula performs the worst among all of models when in terms of backtesting, in particular for the estimation of $\widehat{\text{VaR}}_{0.95}$. By contrast, the Student$-t$ copula performs similarly to the univariate Student$-t$ model, and together they are the models behaving the best, especially for tail expectations and for $\alpha = 0.99$. Overall, in our setting, dependence modelling does not clearly improve the results. A possible explanation is the additional estimation noise introduced by the copula : when we fit the copulas, we estimate three marginal distributions plus the copula parameters that capture cross-asset correlation. These extra parameters can be imprecisely estimated and thereofore become an additional source of noise and errors in the forecasts.

1. Empirical stylized facts.

We first add the code for our implemented functions, and then the application of it

```
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, acf
from statsmodels.tsa.stattools import ccf
from itertools import combinations # plot
import scipy.stats as stats
from scipy.stats import norm
import statsmodels.api as sm
from scipy.stats import jarque_bera


def plot_rolling_vol(returns, window=21):
    ''' Plot rolling window volatility of the three assets'''
    assets = returns.columns
    fig, axes = plt.subplots(len(assets), 1, figsize=(12, 8), sharex=True)

    for ax, asset in zip(axes, assets):
        vol = returns[asset].rolling(window).std()
        vol = vol * np.sqrt(252) # annualized vol

        ax.plot(vol.index, vol, label=f'{asset} {window}-day rolling vol')
        ax.set_ylabel('volatility')
        ax.grid(True)
        ax.legend(loc='upper right')

    axes[-1].set_xlabel('date')
    fig.suptitle(f'{window}-day rolling volatility')
    plt.tight_layout()

def cross_corr(col1, col2):
    ''' compute cross correlation given two returns series'''
    col1 = col1.dropna()
    abs_col1 = np.abs(col1).dropna()

    col2 = col2.dropna()
    abs_col2 = np.abs(col2).dropna()

    cc1 = ccf(col1, col2, adjusted=False, nlags=25)
    cc2 = ccf(abs_col1, abs_col2, adjusted=False, nlags=25)
    return cc1, cc2
```

```python
def cross_corr_plot(prices_log):
    ''' Function that plots the cross correlations of the three stock returns
    and absolute returns. thwo figures with 9 subplots in each'''
    fig1, axs1 = plt.subplots(3,3, figsize=(15,12), sharex=True) # raw
    fig1.suptitle('raw correlation', fontsize=20, y=1.02)
    fig2, axs2 = plt.subplots(3,3, figsize=(15, 12), sharex=True) # abs
    fig2.suptitle('abs correlation', fontsize=20, y=1.02)
    for i, asset1 in enumerate(prices_log.columns):
        for j, asset2 in enumerate(prices_log.columns):
            col1 = prices_log[asset1]
            col2 = prices_log[asset2]
            cc1_raw, cc1_abs = cross_corr(col1, col2)
            lags=np.arange(len(cc1_raw))

            # raw
            axs1[i,j].stem(lags, cc1_raw, label=f'{asset1} | {asset2}')
            axs1[i,j].set_title(f"{asset1} | laggeg {asset2}")

            # abs
            axs2[i, j].stem(lags, cc1_abs, label=f'{asset1} | {asset2}')
            axs2[i, j].set_title(f'{asset1} | lagged {asset2}')

            if i == 0:
                axs1[i,j].set_title(f'Lagged: {asset2}', fontsize=14)
                axs2[i,j].set_title(f'Lagged: {asset2}', fontsize=14)
            if j == 0:
                axs1[i,j].set_ylabel(f'{asset1}', fontsize=14)
                axs2[i,j].set_ylabel(f'{asset1}', fontsize=14)

            if i == 2:
                axs1[i,j].set_xlabel('Lag')
                axs2[i,j].set_xlabel('Lag')
            axs1[i,j].grid(True)
            axs2[i,j].grid(True)

    fig1.tight_layout()
    fig2.tight_layout()


# 3) QQ-plot
def norm_dist(data):
    return norm(loc=data.mean(), scale=data.std())
```

```python
def plot_qq(data):
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    for i, col in enumerate(data.columns):
        sm.qqplot(data[col].dropna(), dist=stats.norm, fit=True, line='45',
                                ax=axes[i])
        axes[i].set_title(f"QQ Plot (standardized) for {col}")
    plt.tight_layout()

# 3) Jarque-Bera test

def JB_test(data):
    stat, pval = jarque_bera(data)
    print(f"statistic : {stat:.4f} | p-val : {pval:.4f}")

    if pval < 0.05:
        print("reject H_0 : data is not normally distributed")
    else : print("fail to reject H_0 : cannot conclude non-normality")

# a) log-returns
prices_log = np.log(prices / prices.shift(1))

# subplots
fig, ax = plt.subplots(3, 1, sharex=True, figsize=(10, 6))

for i in range(3):
    ax[i].plot(prices_log.iloc[:, i])
    ax[i].set_title(prices_log.columns[i])
    ax[i].grid(True, which='both', linestyle='--', alpha=0.5)
    ax[i].set_ylabel('log return')

ax[2].set_xlabel("Date")
fig.tight_layout()

# rolling volatility
plot_rolling_vol(prices_log[['AAPL', 'META', 'JPM']],
    window=21) # since 21 days ~ 1 trading month

# b) cross-correlation
cross_corr_plot(prices_log)

# c) QQ-plot
```

```
plot_qq(prices_log)

# c) Jarque-Bera test
for i in range(3):
    JB_test(prices_log.iloc[:, i].dropna())
```

2. Single-window modeling: VaR, ES, and distributions.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm, t
from statsmodels.tsa.ar_model import AutoReg
from arch import arch_model

# model a)
def historical_stats(loss, alphas):
    x = np.sort(np.asarray(loss))
    n = len(x)
    var_list = []
    es_list = []
    for alpha in alphas:
        var = np.quantile(loss, alpha)
        es = x[x>=var].mean()
        var_list.append(var)
        es_list.append(es)

    return var_list, es_list

# model b)
def gaussian_model(loss, alphas):
    mu, sigma = norm.fit(loss)
    var_list = []; es_list = []
    for alpha in alphas:
        var_g = mu + sigma * norm.ppf(alpha)
        var_list.append(var_g)
        es_g = mu + sigma/(1-alpha) * norm.pdf(norm.ppf(alpha))
        es_list.append(es_g)
    return mu, sigma, var_list, es_list

# model c)
def student_model(loss, alphas):
    nu, mu, sigma = t.fit(loss)
    var_list = []; es_list = []
    for alpha in alphas:
        q = t.ppf(alpha, nu)
        dens = t.pdf(q, nu)
        var_t = mu + sigma * q
        var_list.append(var_t)
        es_t = mu + sigma * ((nu + q**2)/((nu-1) * (1-alpha))) * dens
        es_list.append(es_t)
```

```
    return nu, mu, sigma, var_list, es_list


def arch_garch(loss, alphas, p):
    # AR(p)
    if p > 0:
        ar_model = AutoReg(loss, lags=p).fit()
        mean_forecast = ar_model.forecast(steps=p)[0]
        resid = ar_model.resid
    else:
        mean_forecast = loss.mean()
        resid = loss - loss.mean()
        ar_model = None

    resid = resid[~np.isnan(resid)]

    # GARCH(1, 1)
    am = arch_model(resid, mean='Zero', vol='Garch', 1, 1,
        dist='gaussian', rescale=False)
            .fit(disp='off', show_warning=False, tol=1e-7)
    vol_forecast = np.sqrt(am.forecast(horizon=1).variance.values[-1, 0])

    # conditional VaR and ES
    var_list = []; es_list = []
    for alpha in alphas:
        q = norm.ppf(alpha)
        pdf_q = norm.pdf(q)
        var_cond = mean_forecast + vol_forecast * q
        var_list.append(var_cond)
        es_cond = mean_forecast + vol_forecast * (pdf_q / (1 - alpha))
        es_list.append(es_cond)

    # useful for model e)
    if p > 0:
        eps_t = ar_model.resid
    else : eps_t = resid
    sig_t = am.conditional_volatility
    z_t = eps_t / sig_t

    return mean_forecast, vol_forecast, var_list, es_list, z_t

# model e)
```

```python
def fhs_model(alphas, mu_hat, sigma_hat, std_res, M=1000):
    rng = np.random.default_rng(42) # for reprod
    z_boot = rng.choice(np.asarray(std_res), size=M, replace=True)
    L_sim = mu_hat + sigma_hat * z_boot
    var_list = []; es_list = []
    L_sorted = np.sort(L_sim)

    for alpha in alphas:
        var = np.quantile(L_sorted, alpha)
        var_list.append(var)
        es = L_sorted[L_sorted >= var].mean()
        es_list.append(es)

    return var_list, es_list, L_sim

from utils.part2 import historical_stats, gaussian_model, student_model,
    arch_garch, fhs_model
import seaborn as sns
from scipy.stats import norm, t
loss = -prices_log.dropna()
W = 252
window_loss = loss.iloc[:W, :]
prices_log_window = prices_log.iloc[:W, :].dropna()

# plot pacf for model d)
fig, axes = plt.subplots(1, 3, figsize=(15,4))

for i, tick in zip(range(3), TICKERS):
    prices_log_series = prices_log_window[tick].values
    plot_pacf(prices_log_series, lags=25, ax=axes[i])
    axes[i].set_title(f'PACF - {TICKERS[i]}')
    axes[i].set_xlabel('lag')


assets = loss.columns
p = 0 # for model d)

for j, asset in enumerate(assets):
    window_loss = loss[asset].iloc[:W].dropna().values

    # model a)
    var_hist, es_hist = historical_stats(window_loss, ALPHAS)

    # model b)
```

```
mu, sigma, var_g, es_g = gaussian_model(window_loss, ALPHAS)

# model c)
deg_f, mu_t, sigma_t, var_t, es_t = student_model(window_loss, ALPHAS)

# model d)
mean_forecast, vol_forecast, var_cond, es_cond, z_t
    = arch_garch(window_loss, ALPHAS, p)

# model e)
var_fhs, es_fhs, L_sim_fhs = fhs_model(ALPHAS, mean_forecast, vol_forecast, z_t)
```

3. Backtesting VaR and ES.

```
from .part2 import historical_stats, gaussian_model, student_model, arch_garch,
    fhs_model
from scipy.stats import chi2

def oos_stats(loss, alphas, model:str):
    oos_VaR = []; oos_ES = []
    T = loss.shape[0]
    W = 252

    for t in range(W, T):
        window_loss = loss.iloc[t-W: t]
        if model == 'historical':
            var, es = historical_stats(window_loss, alphas)
        elif model == 'gaussian':
            _, _, var, es = gaussian_model(window_loss, alphas)
        elif model == 'student':
            _, _, _, var, es, = student_model(window_loss, alphas)
        elif model == 'arch_garch':
            _, _, var, es, _ = arch_garch(window_loss.values, alphas, p=0)
        elif model == 'fhs':
            mu_hat, sigma_hat, _, _, z_t = arch_garch(window_loss.values, alphas, 0)
            var, es, _ = fhs_model(alphas, mu_hat, sigma_hat, z_t)

        oos_VaR.append(var)
        oos_ES.append(es)

    # construct output
    idx = loss.index[W:]
    cols = [f"VaR_{alpha}" for alpha in alphas] + [f"ES_{alpha}" for alpha in alphas]
    data = np.hstack([np.array(oos_VaR), np.array(oos_ES)])
    out = pd.DataFrame(data, index=idx, columns=cols)
    out['L_realized'] = loss.iloc[W:].values
    return out

def pof_test(df):
    rows = []
    L = df['L_realized'].to_numpy()
    T = df.shape[0]
    var_cols=("VaR_0.95","VaR_0.99")
    alphas=(0.95, 0.99)
    for var_col, alpha in zip(var_cols, alphas):
        q = df[var_col].to_numpy()
```

```
        hits = (L > q).astype(int)
        N = hits.sum()
        p = 1.0 - alpha
        p_hat = max(min(N/T, 1-1e-12), 1e-12)

        LR = -2.0 * ((T-N) * np.log((1-p)/(1-p_hat)) + N * np.log(p/p_hat))
        p_val = 1.0 - chi2.cdf(LR, df=1)

        rows.append(dict(
            alpha=alpha,
            var_col=var_col,
            T = T,
            N = int(N),
            hit_rate = p_hat,
            LR_POF = float(LR),
            p_value = float(p_val),
            reject_pct = bool(p_val < 1-alpha),
            coverage_bias = p_hat - p
        ))

    return pd.DataFrame(rows)

def christoffersen_indep(df):
    rows = []
    L = df['L_realized'].to_numpy()
    T = df.shape[0]
    var_cols=("VaR_0.95","VaR_0.99")
    alphas=(0.95, 0.99)
    epsilon = 1e-12 # numerical stability
    for var_col, alpha in zip(var_cols, alphas):
        q = df[var_col].to_numpy()
        hits = (L > q).astype(int) # I_t
        T_eff = len(hits)

        # transition counts
        N00 = N01 = N10 = N11 = 0
        for t in range(1, T_eff):
            i = hits[t-1]; j = hits[t]
            if i == 0 and j == 0 : N00 += 1
            elif i == 0 and j == 1 : N01 += 1
            elif i == 1 and j == 0 : N10 += 1
            elif i == 1 and j == 1 : N11 += 1
```

```
        denom0 = N00 + N01
        denom1 = N10 + N11
        total = N00 + N01 + N10 + N11

        if total == 0 or denom0 == 0 or denom1 == 0:
            LR_ind = 0.0
            p_ind = 1.0
        else:
            pi0_hat = N01 / denom0
            pi1_hat = N11 / denom1
            p_hat   = (N01 + N11) / total

            # clamp probabilities away from 0 and 1 to avoid log(0)
            pi0_hat = np.clip(pi0_hat, epsilon, 1-epsilon)
            pi1_hat = np.clip(pi1_hat, epsilon, 1-epsilon)
            p_hat   = np.clip(p_hat,   epsilon, 1-epsilon)

            logL1 = 0.0
            if N00 > 0: logL1 += N00 * np.log(1 - pi0_hat)
            if N01 > 0: logL1 += N01 * np.log(pi0_hat)
            if N10 > 0: logL1 += N10 * np.log(1 - pi1_hat)
            if N11 > 0: logL1 += N11 * np.log(pi1_hat)

            logL0 = 0.0
            if (N00 + N10) > 0: logL0 += (N00 + N10) * np.log(1 - p_hat)
            if (N01 + N11) > 0: logL0 += (N01 + N11) * np.log(p_hat)

            LR_ind = -2 * (logL0 - logL1)
            p_ind  = chi2.sf(LR_ind, df=1)

        rows.append(dict(
            alpha = alpha,
            var_col = var_col,
            N00=N00, N01=N01, N10=N10, N11=N11,
            LR_ind=float(LR_ind),
            p_ind=float(p_ind),
            reject_ind = (p_ind < 1-alpha)
        ))
    return pd.DataFrame(rows)

def conditional_coverage(LR_pof, LR_ind):
    LR_cc = LR_pof + LR_ind
    p_cc = chi2.sf(LR_cc, df=2)
```

```
    return LR_cc, p_cc

def print_stats_tests(pof_df, ind_df, stock:str, model:str):
    print(f'\n====={model} | {stock}====')

    for idx in range(len(pof_df)):
        alpha = pof_df.iloc[idx]['alpha']
        a = 0.05
        LR_pof = pof_df.iloc[idx]['LR_POF']
        p_pof = pof_df.iloc[idx]['p_value']
        reject_pof = (p_pof < a)

        LR_ind = ind_df.loc[idx]['LR_ind']
        p_ind = ind_df.loc[idx]['p_ind']
        reject_ind = (p_ind < a)

        LR_cc = LR_pof + LR_ind
        p_cc = chi2.sf(LR_cc, df=2)
        reject_cc = (p_cc < a)

        #here we remove the print code for clarity of the report
    print("="*40)

def simulate_losses(loss, model: str, M: int, T: int, W: int, rng, alphas):
    L_all = np.asarray(loss)
    T = len(L_all)

    L_sim = np.zeros((T-W, M), dtype=float)

    for t in range(W, T):
        window_loss = L_all[t - W:t]
        if model == 'historical':
            sim_t = rng.choice(window_loss, size=M, replace=True)
        elif model == 'gaussian':
            mu, sigma, _, _ = gaussian_model(window_loss, alphas)
            sim_t = rng.normal(mu, sigma, size=M)
        elif model == 'student':
            nu, mu, sigma, _, _ = student_model(window_loss, alphas)
            sim_t = mu + sigma * rng.standard_t(df=nu, size=M)
        elif model == 'arch_garch':
            mean_f, vol_f, _, _, _ = arch_garch(window_loss, alphas, p=0)
            z = rng.normal(size=M)
            sim_t = mean_f + vol_f * z
```

```
        elif model == 'fhs':
            mean_f, vol_f, _, _, z_t = arch_garch(window_loss, alphas, p=0)
            z_t = np.asarray(z_t)
            z_boot = rng.choice(z_t, size=M, replace=True)
            sim_t = mean_f + vol_f * z_boot

        L_sim[t-W, :] = sim_t

    return L_sim


def z1_single_alpha(loss, df, var_col, es_col, model: str,
                    simulate_losses_func, alphas, W=252, M=1000):
    L = df['L_realized'].to_numpy()
    q = df[var_col].to_numpy()
    e = df[es_col].to_numpy()
    T = len(L)

    I = (L > q).astype(int)
    N = I.sum()
    if N == 0:
        return np.nan, np.array([]), np.nan

    Z1_obs = (L * I / e).sum() / N - 1.0

    # Simulate paths
    rng = np.random.default_rng()
    L_sim_full = simulate_losses_func(loss, model, M, T, W, rng, alphas) #(T,M)
    Z1_sim_list = []
    for m in range(M):
        Lm = L_sim_full[:, m]
        Im = (Lm > q).astype(int)
        Nm = Im.sum()
        if Nm == 0:
            continue
        Z1m = (Lm * Im / e).sum() / Nm - 1.0
        Z1_sim_list.append(Z1m)

    Z1_sim = np.array(Z1_sim_list)
    if Z1_sim.size == 0:
        p_val = np.nan
    else:
        p_val = float(np.mean(Z1_sim >= Z1_obs))
```

```
    return Z1_obs, Z1_sim, p_val

def z1_es_test(loss, df, stock, model, simulate_losses_func, M=1000, W=252):
    alphas = (0.95, 0.99)
    var_cols = ('VaR_0.95', 'VaR_0.99')
    es_cols = ('ES_0.95', 'ES_0.99')
    rows = []

    print(f'Z1 $\ES$ backtest | Stock {stock} | Model : {model}')
    for alpha, vcol, ecol in zip(alphas, var_cols, es_cols):
        Z1_obs, Z1_sim, p_val = z1_single_alpha(
            loss=loss,
            df=df,
            var_col=vcol,
            es_col=ecol,
            model=model,
            simulate_losses_func=simulate_losses_func,
            alphas=alphas,
            W=W,
            M=M
        )

        decision = 'Reject H0' if (not np.isnan(p_val) and p_val < 0.05)
            else 'Fail to reject H0'

        rows.append(dict(
            stock=stock,
            model=model,
            alpha=alpha,
            Z1_obs=Z1_obs,
            p_value=p_val,
            decision=decision
        ))
    print('=' * 50)
    return pd.DataFrame(rows)

 from utils.part3 import oos_stats
jpm_log = -prices_log['JPM'].dropna()
meta_log = -prices_log['META'].dropna()
aapl_log = -prices_log['AAPL'].dropna()

# historical simulation
jpm_stats_historical = oos_stats(jpm_log, ALPHAS, model='historical')
```

```
meta_stats_historical = oos_stats(meta_log, ALPHAS, model='historical')
aapl_stats_historical = oos_stats(aapl_log, ALPHAS, model='historical')

# gaussian
jpm_stats_gaussian = oos_stats(jpm_log, ALPHAS, model='gaussian')
meta_stats_gaussian = oos_stats(meta_log, ALPHAS, model='gaussian')
aapl_stats_gaussian = oos_stats(aapl_log, ALPHAS, model='gaussian')

# student approx 10-15s to run
jpm_stats_student = oos_stats(jpm_log, ALPHAS, model='student')
meta_stats_student = oos_stats(meta_log, ALPHAS, model='student')
aapl_stats_student = oos_stats(aapl_log, ALPHAS, model='student')

# ARCH-GARCH
jpm_stats_arch_garch = oos_stats(jpm_log, ALPHAS, model='arch_garch')
meta_stats_arch_garch = oos_stats(meta_log, ALPHAS, model='arch_garch')
aapl_stats_arch_garch = oos_stats(aapl_log, ALPHAS, model='arch_garch')

# FHS
jpm_stats_fhs = oos_stats(jpm_log, ALPHAS, model='fhs')
meta_stats_fhs = oos_stats(meta_log, ALPHAS, model='fhs')
aapl_stats_fhs = oos_stats(aapl_log, ALPHAS, model='fhs')
```

```
from utils.part3 import pof_test, christoffersen_indep, print_stats_tests
# historical
# AAPL
pof_df_aapl = pof_test(aapl_stats_historical)
ind_df_aapl = christoffersen_indep(aapl_stats_historical)
print_stats_tests(pof_df_aapl, ind_df_aapl, stock='AAPL', model='historical')

# META
pof_df_meta = pof_test(meta_stats_historical)
ind_df_meta = christoffersen_indep(meta_stats_historical)
print_stats_tests(pof_df_meta, ind_df_meta, stock='META', model='historical')

# JPM
pof_df_jpm = pof_test(jpm_stats_historical)
ind_df_jpm = christoffersen_indep(jpm_stats_historical)
print_stats_tests(pof_df_jpm, ind_df_jpm, stock='JPM', model='historical')

# gaussian
```

```
# AAPL
pof_df_aapl = pof_test(aapl_stats_gaussian)
ind_df_aapl = christoffersen_indep(aapl_stats_gaussian)
print_stats_tests(pof_df_aapl, ind_df_aapl, stock='AAPL', model='gaussian')

# META
pof_df_meta = pof_test(meta_stats_gaussian)
ind_df_meta = christoffersen_indep(meta_stats_gaussian)
print_stats_tests(pof_df_meta, ind_df_meta, stock='META', model='gaussian')

# JPM
pof_df_jpm = pof_test(jpm_stats_gaussian)
ind_df_jpm = christoffersen_indep(jpm_stats_gaussian)
print_stats_tests(pof_df_jpm, ind_df_jpm, stock='JPM', model='gaussian')


# student
# AAPL
pof_df_aapl = pof_test(aapl_stats_student)
ind_df_aapl = christoffersen_indep(aapl_stats_student)
print_stats_tests(pof_df_aapl, ind_df_aapl, stock='AAPL', model='student')

# META
pof_df_meta = pof_test(meta_stats_student)
ind_df_meta = christoffersen_indep(meta_stats_student)
print_stats_tests(pof_df_meta, ind_df_meta, stock='META', model='student')

# JPM
pof_df_jpm = pof_test(jpm_stats_student)
ind_df_jpm = christoffersen_indep(jpm_stats_student)
print_stats_tests(pof_df_jpm, ind_df_jpm, stock='JPM', model='student')


# ARCH-GARCH
# JPM
pof_df_jpm = pof_test(jpm_stats_arch_garch)
ind_df_jpm = christoffersen_indep(jpm_stats_arch_garch)
print_stats_tests(pof_df_jpm, ind_df_jpm, stock='JPM',
    model='conditional parametric')

# META
pof_df_meta = pof_test(meta_stats_arch_garch)
ind_df_meta = christoffersen_indep(meta_stats_arch_garch)
```

```
print_stats_tests(pof_df_meta, ind_df_meta, stock='META',
    model='conditional parametric')

# AAPL
pof_df_aapl = pof_test(aapl_stats_arch_garch)
ind_df_aapl = christoffersen_indep(aapl_stats_arch_garch)
print_stats_tests(pof_df_aapl, ind_df_aapl, stock='AAPL',
    model='conditional parametric')


# FHS
# JPM
pof_df_jpm = pof_test(jpm_stats_fhs)
ind_df_jpm = christoffersen_indep(jpm_stats_fhs)
print_stats_tests(pof_df_jpm, ind_df_jpm, stock='JPM', model='fhs')

# META
pof_df_meta = pof_test(meta_stats_fhs)
ind_df_meta = christoffersen_indep(meta_stats_fhs)
print_stats_tests(pof_df_meta, ind_df_meta, stock='META', model='fhs')

# AAPL
pof_df_aapl = pof_test(aapl_stats_fhs)
ind_df_aapl = christoffersen_indep(aapl_stats_fhs)
print_stats_tests(pof_df_aapl, ind_df_aapl, stock='AAPL', model='fhs')


# $\ES$ backtest
from utils.part3 import z1_es_test, simulate_losses

# historical
z1_es_aapl = z1_es_test(aapl_log, aapl_stats_historical, stock='AAPL',
    model='historical', simulate_losses_func=simulate_losses)
z1_es_meta = z1_es_test(meta_log, meta_stats_historical, stock='META',
    model='historical', simulate_losses_func=simulate_losses)
z1_es_jpm = z1_es_test(jpm_log, jpm_stats_historical, stock='JPM',
    model='historical', simulate_losses_func=simulate_losses)

# gaussian
z1_es_aapl = z1_es_test(aapl_log, aapl_stats_gaussian, stock='AAPL',
    model='gaussian', simulate_losses_func=simulate_losses)
z1_es_meta = z1_es_test(meta_log, meta_stats_gaussian, stock='META',
    model='gaussian', simulate_losses_func=simulate_losses)
```

```
z1_es_jpm = z1_es_test(jpm_log, jpm_stats_gaussian, stock='JPM',
    model='gaussian', simulate_losses_func=simulate_losses)

# student
z1_es_aapl = z1_es_test(aapl_log, aapl_stats_student, stock='AAPL',
    model='student', simulate_losses_func=simulate_losses)
z1_es_meta = z1_es_test(meta_log, meta_stats_student, stock='META',
    model='student', simulate_losses_func=simulate_losses)
z1_es_jpm = z1_es_test(jpm_log, jpm_stats_student, stock='JPM',
    model='student', simulate_losses_func=simulate_losses)

# ARCH-GARCH
z1_es_aapl = z1_es_test(aapl_log, aapl_stats_arch_garch, stock='AAPL',
    model='arch_garch', simulate_losses_func=simulate_losses)
z1_es_meta = z1_es_test(meta_log, meta_stats_arch_garch, stock='META',
    model='arch_garch', simulate_losses_func=simulate_losses)
z1_es_jpm = z1_es_test(jpm_log, jpm_stats_arch_garch, stock='JPM',
    model='arch_garch', simulate_losses_func=simulate_losses)

# FHS
z1_es_aapl = z1_es_test(aapl_log, aapl_stats_fhs, stock='AAPL',
    model='fhs', simulate_losses_func=simulate_losses)
z1_es_meta = z1_es_test(meta_log, meta_stats_fhs, stock='META',
    model='fhs', simulate_losses_func=simulate_losses)
z1_es_jpm = z1_es_test(jpm_log, jpm_stats_fhs, stock='JPM',
    model='fhs', simulate_losses_func=simulate_losses)
```

4. Copula fitting.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

def scatter_return(R_i, R_j, ax=None):
    if ax is None:
        fig, ax = plt.subplots(figsize=(5,5))
    ax.scatter(R_i, R_j, alpha=0.6)
    ax.set_title(f'{R_i.name} vs {R_j.name} - Raw Returns')
    ax.set_xlabel(f'{R_i.name} log-return')
    ax.set_ylabel(f'{R_j.name} log-return')
    ax.grid(True, alpha=0.3)
    return ax

def scatter_pseudo(U_i, U_j, ax=None):
    if ax is None:
        fig, ax = plt.subplots(figsize=(5,5))
    ax.scatter(U_i, U_j, alpha=0.6)
    ax.set_title(f'{U_i.name} vs {U_j.name} - Pseudo-observations')
    ax.set_xlabel(f'U_{U_i.name}')
    ax.set_ylabel(f'U_{U_j.name}')
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.grid(True, alpha=0.3)
    return ax

def inv_cdf(u, data):
    data_sorted = np.sort(data)
    n = 252
    u_clipped = np.clip(u, 1/(n+1), n/(n+1)) # empirical quantiles of unif dist
    return np.quantile(data_sorted, u_clipped)

def density_return(R_true, R_gauss, R_stud, ax=None):
    if ax is None:
        fig, ax = plt.subplots(figsize=(5,4))

    sns.kdeplot(R_true, ax=ax, lw=2, label='Empirical')
    sns.kdeplot(R_gauss, ax=ax, lw=1.5, label='Gaussian Copula')
    sns.kdeplot(R_stud, ax=ax, lw=1.5, label='t-Copula')

    ax.set_title(f"{R_true.name} - Marginal Density")
    ax.set_xlabel(f"{R_true.name} log-return")
```

```
    ax.set_ylabel("Density")
    ax.grid(alpha=0.3)
    ax.legend()

    return ax

def compare_timeseries(df, sim_gauss, sim_stud, stock:str):
    true_ret = df[stock]
    idx = true_ret.index
    fig, ax = plt.subplots(1,1, figsize=(9, 4))
    ax.plot(idx, true_ret, label='true returns', linewidth=1)
    ax.plot(idx, sim_gauss, label='Gaussian copula', linewidth=1)
    ax.plot(idx, sim_stud, label='Student-t copula', linewidth=1)

    ax.set_title(f'{stock} : true vs simulated returns')
    ax.set_xlabel('date')
    ax.set_ylabel('return')
    ax.legend()
    plt.tight_layout()

from scipy.stats import rankdata
jpm = prices_log['JPM'].dropna()[:W]
U_jpm = pd.Series(rankdata(jpm.values, method='average') / (W + 1),
                  index=jpm.index, name='JPM')

meta = prices_log['META'].dropna()[:W]
U_meta = pd.Series(rankdata(meta.values, method='average') / (W + 1),
                   index=meta.index, name='META')

aapl = prices_log['AAPL'].dropna()[:W]
U_aapl = pd.Series(rankdata(aapl.values, method='average') / (W + 1),
                   index=aapl.index, name='AAPL')

# a) scatter plot of (R_i, R_j)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
scatter_return(aapl, meta, ax=axes[0])
scatter_return(jpm, aapl, ax=axes[1])
scatter_return(meta, jpm, ax=axes[2])
plt.tight_layout()

# a) scatter plot of (U_i, U_j)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
scatter_pseudo(U_aapl, U_meta, ax=axes[0])
scatter_pseudo(U_jpm, U_aapl, ax=axes[1])
```

```
scatter_pseudo(U_meta, U_jpm, ax=axes[2])
plt.tight_layout()

# b) copulas
from copulae import GaussianCopula, StudentCopula

U_data = pd.DataFrame({
    'JPM': U_jpm,
    'META' : U_meta,
    'AAPL' : U_aapl
})

# b) Gaussian Copula

gauss_cop = GaussianCopula(dim=3)
gauss_cop.fit(U_data) # Estimated Correlation Matrix
gauss_params = gauss_cop.params

print(f"Corr(JPM, META) = {gauss_params[0]:.4f}")
print(f"Corr(JPM, AAPL) = {gauss_params[1]:.4f}")
print(f"Corr(META, AAPL) = {gauss_params[2]:.4f}")

# b) Student Copula

stud_cop = StudentCopula(dim=3)
stud_cop.fit(U_data)
stud_params = stud_cop.params

print(f"Degrees of freedom : {stud_params[0]:.4f}")
print(f"Corr(JPM, META) = {stud_params[1][0]:.4f}")
print(f"Corr(JPM, AAPL) = {stud_params[1][1]:.4f}")
print(f"Corr(META, AAPL) = {stud_params[1][2]:.4f}")

# c)

from utils.part4 import inv_cdf, density_return
T = prices_log.shape[0]

# Gaussian copula simulation
U_sim_gauss = gauss_cop.random(T)
jpm_sim_gauss = inv_cdf(U_sim_gauss.iloc[:,0], jpm)
meta_sim_gauss = inv_cdf(U_sim_gauss.iloc[:,1], meta)
aapl_sim_gauss = inv_cdf(U_sim_gauss.iloc[:,2], aapl)
```

```
# Student-t copula simulation
U_sim_stud  = stud_cop.random(T)
jpm_sim_stud = inv_cdf(U_sim_stud.iloc[:,0], jpm)
meta_sim_stud = inv_cdf(U_sim_stud.iloc[:,1], meta)
aapl_sim_stud = inv_cdf(U_sim_stud.iloc[:,2], aapl)

# plot
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

density_return(jpm,  jpm_sim_gauss,  jpm_sim_stud,  ax=axes[0])
density_return(meta, meta_sim_gauss, meta_sim_stud, ax=axes[1])
density_return(aapl, aapl_sim_gauss, aapl_sim_stud, ax=axes[2])
plt.tight_layout()

from utils.part4 import compare_timeseries

# time series returns comparison
compare_timeseries(prices_log, aapl_sim_gauss, aapl_sim_stud, 'AAPL')
compare_timeseries(prices_log, meta_sim_gauss, meta_sim_stud, 'META')
compare_timeseries(prices_log, jpm_sim_gauss, jpm_sim_stud, 'JPM')
```

5. Backtesting portfolio VaR and ES.

```
import numpy as np
import pandas as pd
from .part4 import inv_cdf
from scipy.stats import rankdata
from copulae.elliptical import GaussianCopula, StudentCopula

def rolling_copula_pf(returns_df, alphas, method:str, W=252, N=1000):
    assets = ['JPM', 'META', 'AAPL']
    data = returns_df[assets].dropna()
    T = data.shape[0]

    pf_ret = data.mean(axis=1)
    pf_loss = -pf_ret

    var_dict = {f"VaR_{alpha:.2f}": [] for alpha in alphas}
    es_dict  = {f"ES_{alpha:.2f}": [] for alpha in alphas}
    L_realized = []
    out_dates = []

    for t in range(W, T):
        window = data.iloc[t-W: t]
        U_cols = []
        for col in assets:
            x = window[col]
            ranks = x.rank(method='first')
            U = ranks / (W+1.0)
            U_cols.append(U)


        U_window = np.column_stack(U_cols)

        if method == 'gaussian':
            # gaussian copula
            cop = GaussianCopula(dim=3)
        if method == 'student':
            # student copula
            cop = StudentCopula(dim=3)

        cop.fit(U_window)

        U_sim = cop.random(N) # shape (N, 3)
        U_sim = np.asarray(U_sim)
```

```
        R_sim = {}
        for j, col in enumerate(assets):
            R_sim[col] = inv_cdf(U_sim[:, j], window[col].to_numpy())

        R_pf_sim = (R_sim['JPM'] + R_sim['META'] + R_sim['AAPL']) / 3.0
        L_pf_sim = -R_pf_sim

        for alpha in alphas:
            col_var = f"VaR_{alpha:.2f}"
            col_es = f"ES_{alpha:.2f}"

            var_alpha = np.quantile(L_pf_sim, alpha)
            var_dict[col_var].append(var_alpha)

            k = max(1, int(np.ceil((1-alpha) * N)))
            es_alpha = np.sort(L_pf_sim)[-k:].mean()
            es_dict[col_es].append(es_alpha)

        L_realized.append(pf_loss.iloc[t])
        out_dates.append(data.index[t])

    res = pd.DataFrame(index=pd.DatetimeIndex(out_dates))
    for alpha in alphas:
        res[f"VaR_{alpha:.2f}"] = var_dict[f"VaR_{alpha:.2f}"]
        res[f"ES_{alpha:.2f}"]  = es_dict[f"ES_{alpha:.2f}"]
    res["L_realized"] = L_realized

    return res


# create portfolio
pf_ret = prices_log[['JPM', 'META', 'AAPL']].dropna().mean(axis=1)
pf_loss = -pf_ret
pf_loss.name = 'PF'

# check AR(p) for portfolio
plot_pacf(pf_ret, lags=25) # again p = 0

# a) univariate models
pf_stats_historical = oos_stats(pf_loss, ALPHAS, model='historical')
pf_stats_gaussian = oos_stats(pf_loss, ALPHAS, model='gaussian')
pf_stats_student = oos_stats(pf_loss, ALPHAS, model='student')
```

```
pf_stats_arch_garch = oos_stats(pf_loss, ALPHAS, model='arch_garch') # implicit p = 0
pf_stats_fhs = oos_stats(pf_loss, ALPHAS, model='fhs')

# tests for portfolios

# historical
pof_pf_historical = pof_test(pf_stats_historical)
ind_pf_historical = christoffersen_indep(pf_stats_historical)
print_stats_tests(pof_pf_historical, ind_pf_historical, stock='Portfolio ',
    model=' Historical')
z1_es_pf = z1_es_test(pf_loss, pf_stats_historical, stock='Portfolio',
    model='historical', simulate_losses_func=simulate_losses)

# gaussian
pof_pf_gaussian = pof_test(pf_stats_gaussian)
ind_pf_gaussian = christoffersen_indep(pf_stats_gaussian)
print_stats_tests(pof_pf_gaussian, ind_pf_gaussian,
stock='Portfolio ',
model=' Gaussian')
z1_es_pf = z1_es_test(pf_loss, pf_stats_gaussian,
    stock='Portfolio', model='gaussian', simulate_losses_func=simulate_losses)

# student
pof_pf_student = pof_test(pf_stats_student)
ind_pf_student = christoffersen_indep(pf_stats_student)
print_stats_tests(pof_pf_student, ind_pf_student, stock='Portfolio ',
    model=' student')
z1_es_pf = z1_es_test(pf_loss, pf_stats_student, stock='Portfolio',
    model='student', simulate_losses_func=simulate_losses)

# ARCH-GARCH
pof_pf_arch_garch = pof_test(pf_stats_arch_garch)
ind_pf_arch_garch = christoffersen_indep(pf_stats_arch_garch)
print_stats_tests(pof_pf_arch_garch, ind_pf_arch_garch, stock='Portfolio ',
    model=' ARCH-GARCH')
z1_es_pf = z1_es_test(pf_loss, pf_stats_arch_garch, stock='Portfolio',
    model='arch_garch', simulate_losses_func=simulate_losses)

# FHS
pof_pf_fhs = pof_test(pf_stats_fhs)
ind_pf_fhs = christoffersen_indep(pf_stats_fhs)
print_stats_tests(pof_pf_fhs, ind_pf_historical, stock='Portfolio ',
    model='fhs')
```

```
z1_es_pf = z1_es_test(pf_loss, pf_stats_fhs, stock='Portfolio', model='fhs',
    simulate_losses_func=simulate_losses)


# b) copulas portfolio
from utils.part5 import rolling_copula_pf

# gaussian
pf_gauss_cop = rolling_copula_pf(prices_log[['JPM','META','AAPL']],
    alphas=(0.95, 0.99), method='gaussian', W=252, N=1000)

# student
pf_student_cop = rolling_copula_pf(prices_log[['JPM','META','AAPL']],
    alphas=(0.95, 0.99), method='gaussian', W=252, N=1000)


# c) backtesting copula portfolios

# gaussian backtesting
pof_pf_gauss_cop = pof_test(pf_gauss_cop)
ind_pf_gauss_cop = christoffersen_indep(pf_gauss_cop)
print_stats_tests(pof_pf_gauss_cop, ind_pf_gauss_cop,
    stock='Copula Portfolio', model='gaussian')
z1_es_pf_gauss_cop = z1_es_test(pf_loss, pf_gauss_cop,
    stock='Copula Portfolio', model='gaussian',
        simulate_losses_func=simulate_losses)

# student backtesting
pof_pf_stud_cop = pof_test(pf_student_cop)
ind_pf_stud_cop = christoffersen_indep(pf_student_cop)
print_stats_tests(pof_pf_stud_cop, ind_pf_stud_cop,
    stock='Copula Portfolio', model='student')
z1_es_pf_student_cop = z1_es_test(pf_loss, pf_student_cop,
    stock='Copula Portfolio', model='student',
        simulate_losses_func=simulate_losses)
```